

When Every Row Becomes a Curve: Using Andrews Transforms for Predictive Truck Maintenance

Turning 170 anonymized sensor readings into visual signatures that detect air pressure system failures before they strand a truck

Simon Müller

```
library(fdars)
library(ggplot2)
library(dplyr)
library(patchwork)
library(knitr)
theme_set(theme_minimal(base_size = 13))
set.seed(42)
```

The Predictive Maintenance Challenge

A fleet maintenance engineer at Scania faces a daily decision: which trucks need workshop attention *before* a breakdown happens? Each truck generates **170 sensor readings** — pressures, temperatures, counters, histogram distributions from the Air Pressure System (APS) and related components. The APS powers the brakes, gear changes, and clutch assistance in heavy-duty trucks. When it fails on the road, the consequences are severe: the truck is immobilized (air brakes are fail-safe by design), requiring roadside assistance, towing, delayed deliveries, and potential safety hazards.

The economics are stark. Scania's own cost model assigns:

- **\$10** for an unnecessary workshop inspection (false positive: you check a truck that didn't need it)
- **\$500** for a missed failure (false negative: the APS fails in the field)

A 50:1 cost asymmetry. Missing one real failure costs as much as 50 unnecessary inspections. Any analytical approach must respect this — accuracy alone is meaningless when the cost of errors is this lopsided.

The dataset we'll work with comes from the [UCI Machine Learning Repository](#) (Scania CV AB, IDA 2016 Industrial Challenge). It contains 76,000 truck observations with 170 anonymized sensor features and a binary label: **positive** (failure is APS-related) or **negative** (failure is something else). Feature names like `aa_000`, `ag_005`, `cs_007` hide the actual physical measurements for proprietary reasons.

The challenge: 170 columns, heavy missing data (some features are 80% missing), and a 59:1 class imbalance in the training set. No analyst can “see” a truck’s health in a table of 170 numbers. And standard per-sensor threshold monitoring misses failures that emerge from *combinations* of sensors, not individual extremes.

What if each truck became a single curve — a visual fingerprint encoding all 170 sensor readings simultaneously? That’s what Andrews curves deliver. And once you have curves, an entire toolkit of functional data analysis methods becomes available: outlier detection that classifies the *type* of anomaly, clustering that reveals natural groupings, hypothesis testing that quantifies whether failure-bound trucks genuinely differ, and monitoring bands that flag at-risk trucks before breakdown.

Here’s the pipeline we’ll build:

Step	Method	Business Outcome
Transform	Andrews curves	One visual signature per truck
Validate	Distance preservation proof	Guarantees results map back to sensors
Detect anomalies	Depth-ponderation, magnitude-shape, outliergram	Flag unusual trucks for inspection
Classify anomalies	Magnitude vs shape decomposition	Triage: re-test sensor vs investigate truck
Cluster	K-means + fuzzy c-means	Discover failure subtypes; identify borderline trucks
Test	Permutation test + bootstrap CIs	Quantify how APS-failure profiles differ
Reduce dimensions	FPCA	Identify which sensor groups carry most information
Monitor	Prediction bands + functional boxplot	Ongoing fleet health dashboard

```
# Download and load the Scania APS dataset from UCI
# The files have ~20 metadata rows to skip
url_train <- "https://archive.ics.uci.edu/ml/machine-learning-databases/00421/aps_failure_tr
url_test  <- "https://archive.ics.uci.edu/ml/machine-learning-databases/00421/aps_failure_tes
```

```
# Try loading from cache first, download if needed
cache_dir <- file.path(tempdir(), "scania_cache")
dir.create(cache_dir, showWarnings = FALSE)
train_cache <- file.path(cache_dir, "aps_train.csv")
test_cache <- file.path(cache_dir, "aps_test.csv")

if (!file.exists(train_cache)) {
  download.file(url_train, train_cache, quiet = TRUE)
}
if (!file.exists(test_cache)) {
  download.file(url_test, test_cache, quiet = TRUE)
}

# Read with appropriate settings: skip metadata rows, handle "na" strings
raw_train <- read.csv(train_cache, skip = 20, na.strings = "na",
                      stringsAsFactors = FALSE)
raw_test <- read.csv(test_cache, skip = 20, na.strings = "na",
                     stringsAsFactors = FALSE)

cat(sprintf("Training set: %d trucks x %d columns\n",
            nrow(raw_train), ncol(raw_train)))
```

Training set: 60000 trucks x 171 columns

```
cat(sprintf("Test set:      %d trucks x %d columns\n",
            nrow(raw_test), ncol(raw_test)))
```

Test set: 16000 trucks x 171 columns

```
cat(sprintf("\nClass distribution (training):\n"))
```

Class distribution (training):

```
print(table(raw_train$class))
```

```
neg  pos
59000 1000
```

```
cat(sprintf("\nClass imbalance ratio: %.0f:1\n",
           sum(raw_train$class == "neg") / sum(raw_train$class == "pos")))
```

Class imbalance ratio: 59:1

Data Preparation

The raw data presents three practical challenges: heavy missingness, high dimensionality, and extreme class imbalance. We address each systematically.

```
# Combine train + test for a broader sample, then subsample
raw_all <- rbind(raw_train, raw_test)

# Step 1: Remove features with >40% missing
feature_cols <- setdiff(names(raw_all), "class")
miss_pct <- colMeans(is.na(raw_all[, feature_cols]))
keep_features <- names(miss_pct[miss_pct <= 0.40])
cat(sprintf("Features with <=40%% missing: %d / %d\n",
           length(keep_features), length(feature_cols)))
```

Features with <=40% missing: 161 / 170

```
# Step 2: Impute remaining missing values with column medians
X_raw <- as.matrix(raw_all[, keep_features])
for (j in seq_len(ncol(X_raw))) {
  na_idx <- is.na(X_raw[, j])
  if (any(na_idx)) {
    X_raw[na_idx, j] <- median(X_raw[, j], na.rm = TRUE)
  }
}

# Step 3: Remove zero-variance features
var_vals <- apply(X_raw, 2, var)
keep_var <- var_vals > 1e-10
X_raw <- X_raw[, keep_var]
keep_features <- keep_features[keep_var]
cat(sprintf("Features after removing zero-variance: %d\n", ncol(X_raw)))
```

Features after removing zero-variance: 160

```

failure <- factor(raw_all$class, levels = c("neg", "pos"),
                 labels = c("Normal", "APS Failure"))

# Step 4: Feature selection - rank by Wilcoxon test statistic (nonparametric)
# This is the key step: not all 120+ sensors discriminate failures from normals.
# We select the top sensors to make Andrews curves more focused.
pos_mask <- failure == "APS Failure"
wilcox_p <- numeric(ncol(X_raw))
for (j in seq_len(ncol(X_raw))) {
  wt <- suppressWarnings(wilcox.test(X_raw[pos_mask, j], X_raw[!pos_mask, j]))
  wilcox_p[j] <- wt$p.value
}
names(wilcox_p) <- keep_features

# Keep sensors with p < 0.001 (strongly discriminating), capped at top 30
# for visual clarity and computational speed
sig_features <- names(sort(wilcox_p))[wilcox_p[names(sort(wilcox_p))] < 0.001]
n_select <- min(30, length(sig_features))
top_features <- names(sort(wilcox_p))[1:n_select]

cat(sprintf("\nFeature selection (Wilcoxon rank-sum test):\n"))

```

Feature selection (Wilcoxon rank-sum test):

```
cat(sprintf("  Sensors with p < 0.001: %d\n", length(sig_features)))
```

Sensors with p < 0.001: 143

```
cat(sprintf("  Selected top %d sensors for Andrews curves\n", n_select))
```

Selected top 30 sensors for Andrews curves

```

# Order features by discriminating power (most discriminating first)
# This puts the strongest signals into the lowest Fourier frequencies,
# maximizing visual separation in Andrews curves
X_raw <- X_raw[, top_features]
keep_features <- top_features

```

```

# Step 5: Stratified subsample for computational feasibility
# Keep all APS failures, sample from normals
pos_idx <- which(failure == "APS Failure")
neg_idx <- which(failure == "Normal")

n_neg_sample <- min(1000, length(neg_idx))

set.seed(42)
neg_sample <- sample(neg_idx, n_neg_sample)
sample_idx <- sort(c(pos_idx, neg_sample))

X_sub <- X_raw[sample_idx, ]
failure_sub <- failure[sample_idx]

# Step 6: Train/holdout split (stratified) for monitoring evaluation
set.seed(123)
pos_sub_idx <- which(failure_sub == "APS Failure")
neg_sub_idx <- which(failure_sub == "Normal")

# 70% train, 30% holdout
train_pos <- sort(sample(pos_sub_idx, round(0.7 * length(pos_sub_idx))))
train_neg <- sort(sample(neg_sub_idx, round(0.7 * length(neg_sub_idx))))
train_idx <- sort(c(train_pos, train_neg))
holdout_idx <- setdiff(seq_len(nrow(X_sub)), train_idx)

cat(sprintf("\nWorking sample: %d trucks (%d APS failures + %d normal)\n",
            length(sample_idx), sum(failure_sub == "APS Failure"),
            sum(failure_sub == "Normal"))))

```

Working sample: 2375 trucks (1375 APS failures + 1000 normal)

```

cat(sprintf("  Training set:  %d trucks (%d failures)\n",
            length(train_idx), sum(failure_sub[train_idx] == "APS Failure")))

```

Training set: 1662 trucks (962 failures)

```

cat(sprintf("  Holdout set:   %d trucks (%d failures)\n",
            length(holdout_idx), sum(failure_sub[holdout_idx] == "APS Failure")))

```

Holdout set: 713 trucks (413 failures)

```
# Step 7: Scale using training set statistics (prevent leakage)
train_means <- colMeans(X_sub[train_idx, ])
train_sds <- apply(X_sub[train_idx, ], 2, sd)
train_sds[train_sds < 1e-10] <- 1

X <- scale(X_sub, center = train_means, scale = train_sds)
p <- ncol(X)
sensor_names <- colnames(X)

cat(sprintf("Final feature matrix: %d trucks x %d sensors (ordered by discriminating power)\n",
           nrow(X), p))
```

Final feature matrix: 2375 trucks x 30 sensors (ordered by discriminating power)

```
df_features <- data.frame(
  Sensor = factor(top_features, levels = rev(top_features)),
  neg_log_p = -log10(wilcox_p[top_features])
)

ggplot(df_features, aes(x = Sensor, y = neg_log_p)) +
  geom_col(fill = "steelblue") +
  geom_hline(yintercept = -log10(0.001), linetype = "dashed", color = "red") +
  coord_flip() +
  labs(title = "Feature Selection: Top 30 Discriminating Sensors",
       subtitle = "Ordered by Wilcoxon rank-sum p-value (red line = p = 0.001)",
       x = NULL, y = expression(-log[10](p))) +
  annotate("text", x = 3, y = -log10(0.001) + 2,
         label = "p = 0.001", color = "red", size = 3.5)
```

Feature Selection: Top 30 Discriminating Sensors

Ordered by Wilcoxon rank-sum p-value (red line = $p = 0.001$)

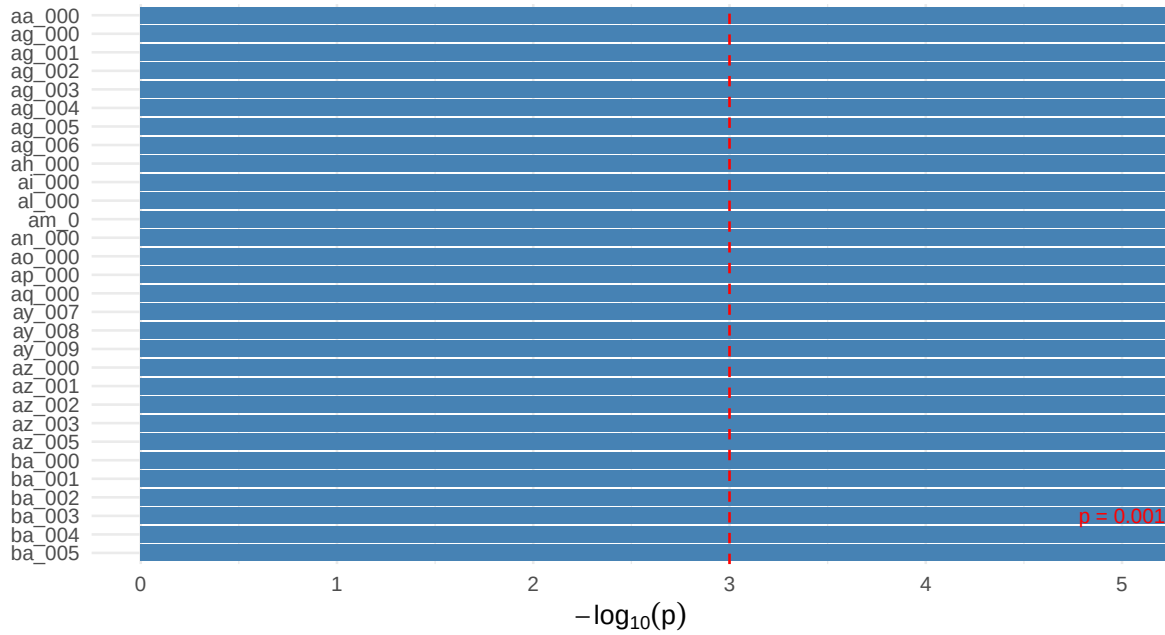


Figure 1: Top 30 sensors ranked by Wilcoxon test p-value. The most discriminating sensors enter the lowest Fourier frequencies in Andrews curves.

```
data.frame(
  Class = c(failure_sub[train_idx], failure_sub[holdout_idx]),
  Split = rep(c("Training", "Holdout"),
              c(length(train_idx), length(holdout_idx)))
) |>
  ggplot(aes(x = Class, fill = Class)) +
  geom_bar() +
  geom_text(stat = "count", aes(label = after_stat(count)), vjust = -0.3) +
  scale_fill_manual(values = c("Normal" = "steelblue", "APS Failure" = "firebrick")) +
  facet_wrap(~ Split) +
  labs(title = "Truck Failure Classes by Split",
       y = "Count", x = NULL) +
  theme(legend.position = "none")
```

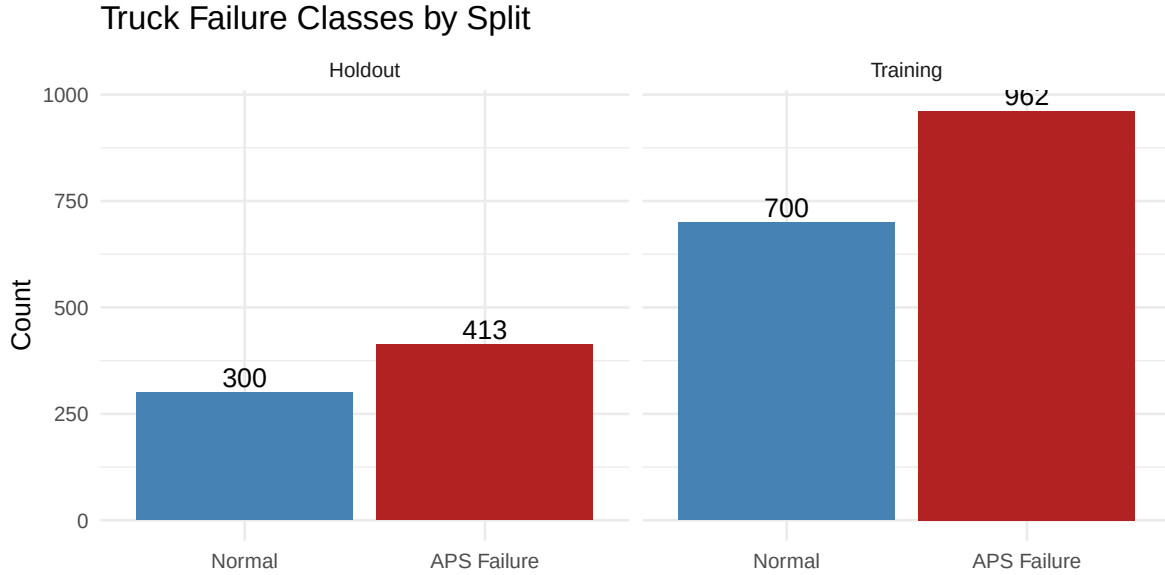



Figure 2: Class distribution: training set (for building references) and holdout set (for evaluating detection).

Turning Sensor Readings into Curves

The Andrews transform maps each truck’s p -dimensional sensor vector $\mathbf{x} = (x_1, x_2, \dots, x_p)$ into a curve $f_x(t)$ on $[-\pi, \pi]$:

$$f_x(t) = \frac{x_1}{\sqrt{2}} + x_2 \sin(t) + x_3 \cos(t) + x_4 \sin(2t) + x_5 \cos(2t) + \dots$$

Each sensor maps to a specific Fourier frequency. Low-order terms (sine and cosine of t , $2t$) capture the broadest patterns; higher-order terms capture finer structure. With 30 sensors, we use 15 Fourier frequencies — enough to encode every sensor reading without loss.

The key mathematical guarantee: **the L^2 distance between two Andrews curves equals $\sqrt{\pi}$ times the Euclidean distance between the original sensor vectors**. Nothing is lost, nothing distorted. Any pattern visible in the curves corresponds to a real pattern in the 170 sensor readings, and vice versa.

```
# Andrews transformation: map p-dimensional rows to curves on [-pi, pi]
andrews_transform <- function(X, m = 200) {
  if (is.data.frame(X)) X <- as.matrix(X)
  n <- nrow(X)
  p <- ncol(X)
```

```

t_grid <- seq(-pi, pi, length.out = m)

# Build Fourier basis matrix: m x p
basis <- matrix(0, m, p)
basis[, 1] <- 1 / sqrt(2)
for (j in 2:p) {
  k <- ceiling((j - 1) / 2)
  if (j %% 2 == 0) {
    basis[, j] <- sin(k * t_grid)
  } else {
    basis[, j] <- cos(k * t_grid)
  }
}

fdata(X %*% t(basis), argvals = t_grid)
}

m <- 200
t_grid <- seq(-pi, pi, length.out = m)
fd_trucks <- andrews_transform(X, m = m)

# Convert to long format for ggplot
fdata_to_long <- function(fd, group = NULL) {
  n <- nrow(fd$data)
  m <- ncol(fd$data)
  df <- data.frame(
    id = rep(1:n, each = m),
    t = rep(fd$argvals, n),
    value = as.numeric(t(fd$data))
  )
  if (!is.null(group)) {
    df$group <- rep(group, each = m)
  }
  df
}

df_curves <- fdata_to_long(fd_trucks, group = failure_sub)

# Sample curves for readability (too many to plot all)
set.seed(42)
plot_pos <- sample(which(failure_sub == "APS Failure"),
  min(100, sum(failure_sub == "APS Failure")))

```

```

plot_neg <- sample(which(failure_sub == "Normal"),
                  min(200, sum(failure_sub == "Normal")))
plot_ids <- sort(c(plot_pos, plot_neg))

df_plot <- df_curves |> filter(id %in% plot_ids)

ggplot(df_plot, aes(x = t, y = value, group = id, color = group)) +
  geom_line(alpha = 0.3, linewidth = 0.3) +
  scale_color_manual(values = c("Normal" = "steelblue", "APS Failure" = "firebrick"),
                    name = "Class") +
  labs(title = "Andrews Curves: 170 Truck Sensors in One View",
       subtitle = sprintf("%d curves shown (blue = normal, red = APS failure)",
                          length(plot_ids)),
       x = expression(t), y = expression(f[x](t))) +
  theme(legend.position = "bottom")

```

Andrews Curves: 170 Truck Sensors in One View

300 curves shown (blue = normal, red = APS failure)

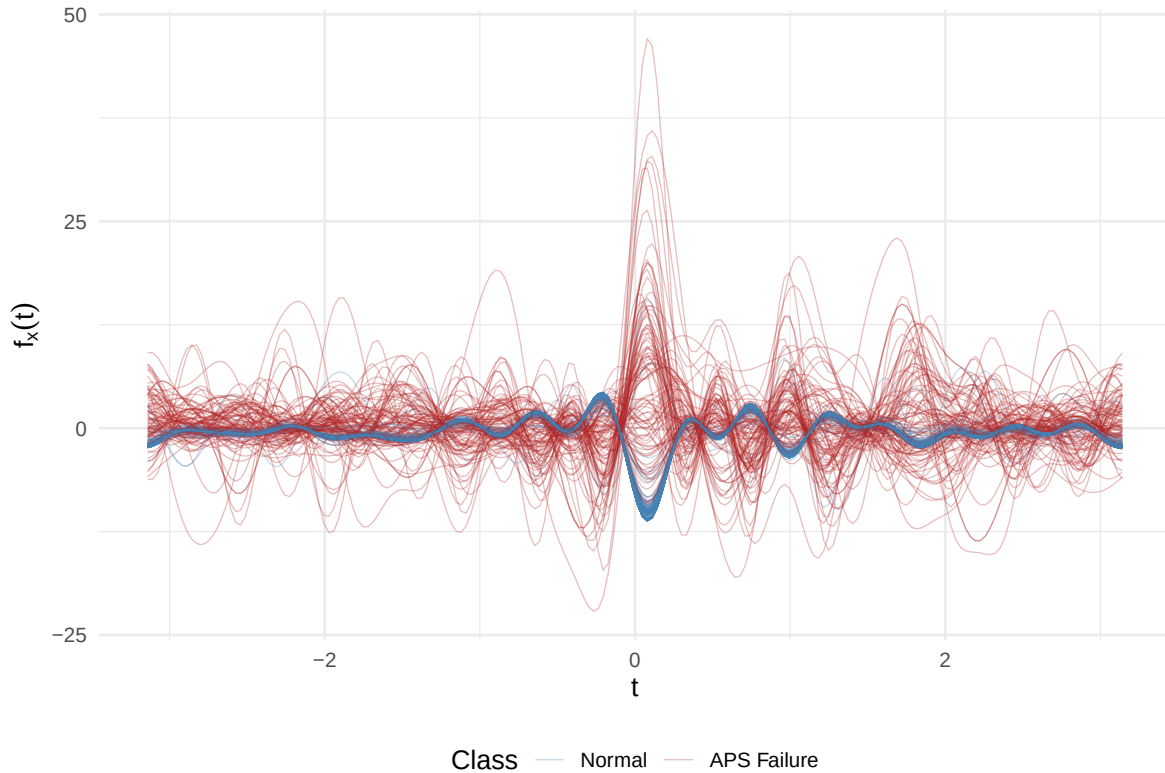


Figure 3: 170 sensor readings per truck, encoded as a single curve. Red = APS failure, blue = normal operation. Failing trucks show systematically different signatures.

Each curve is a **truck fingerprint** — a single visual object encoding all 30 sensor measurements. A fleet engineer can now glance at a curve and compare it to the fleet’s normal profile. Trucks whose curves look different are immediately suspicious. This is the sensor-data equivalent of a chromatography trace — one picture captures the entire system state.

Notice that the APS failure curves (red) tend to separate from normal curves (blue) in certain frequency regions, particularly in the low-order harmonics. This suggests that the most discriminating sensors map to the first few Fourier terms — something we’ll confirm with FPCA later.

Proving the Bridge Works

The distance preservation theorem guarantees that Andrews curves don’t just *look* informative — they are *mathematically equivalent* to the original sensor space. Specifically, for any two

trucks $\mathbf{x}$ and $\mathbf{y}$:

$$\|f_x - f_y\|_{L^2} = \sqrt{\pi} \cdot \|\mathbf{x} - \mathbf{y}\|_2$$

Let's verify this on our data by comparing L^2 curve distances against Euclidean sensor distances for a random sample of truck pairs:

```
# Verify distance preservation on a subsample (full matrix too large)
n_pairs <- 2000
set.seed(42)
idx1 <- sample(nrow(X), n_pairs, replace = TRUE)
idx2 <- sample(nrow(X), n_pairs, replace = TRUE)

# Euclidean distances in original space
euclid_dist <- sqrt(rowSums((X[idx1, ] - X[idx2, ])^2))

# L2 distances in curve space (approximate via trapezoid rule)
dt <- diff(t_grid[1:2])
curve_dist <- sqrt(rowSums((fd_trucks$data[idx1, ] - fd_trucks$data[idx2, ])^2) * dt)

# Theoretical ratio
ratio <- curve_dist / euclid_dist
ratio_clean <- ratio[euclid_dist > 0.1] # filter near-zero distances

cat(sprintf("Distance ratio (curve / Euclidean):\n"))
```

Distance ratio (curve / Euclidean):

```
cat(sprintf("  Mean:    %.4f\n", mean(ratio_clean)))
```

Mean: 1.7766

```
cat(sprintf("  Median:  %.4f\n", median(ratio_clean)))
```

Median: 1.7755

```
cat(sprintf("  Theoretical (sqrt(pi)): %.4f\n", sqrt(pi)))
```

Theoretical (sqrt(pi)): 1.7725

```
cat(sprintf(" Relative error: %.4f%%\n",
           100 * abs(mean(ratio_clean) - sqrt(pi)) / sqrt(pi)))
```

Relative error: 0.2330%

```
df_dist <- data.frame(euclidean = euclid_dist, curve = curve_dist)

ggplot(df_dist |> filter(euclidean > 0.1),
       aes(x = euclidean, y = curve)) +
  geom_point(alpha = 0.1, size = 0.5) +
  geom_abline(intercept = 0, slope = sqrt(pi), color = "red", linewidth = 1) +
  annotate("text", x = max(euclid_dist) * 0.6, y = max(curve_dist) * 0.3,
          label = sprintf("Red line: slope = sqrt(pi) = %.4f", sqrt(pi)),
          color = "red", size = 4) +
  labs(title = "Distance Preservation Verified",
       subtitle = sprintf("Ratio = %.4f (theoretical = %.4f)",
                          mean(ratio_clean), sqrt(pi)),
       x = "Euclidean distance (170 sensors)",
       y = "L2 distance (Andrews curves)") +
  coord_equal(ratio = 1 / sqrt(pi))
```

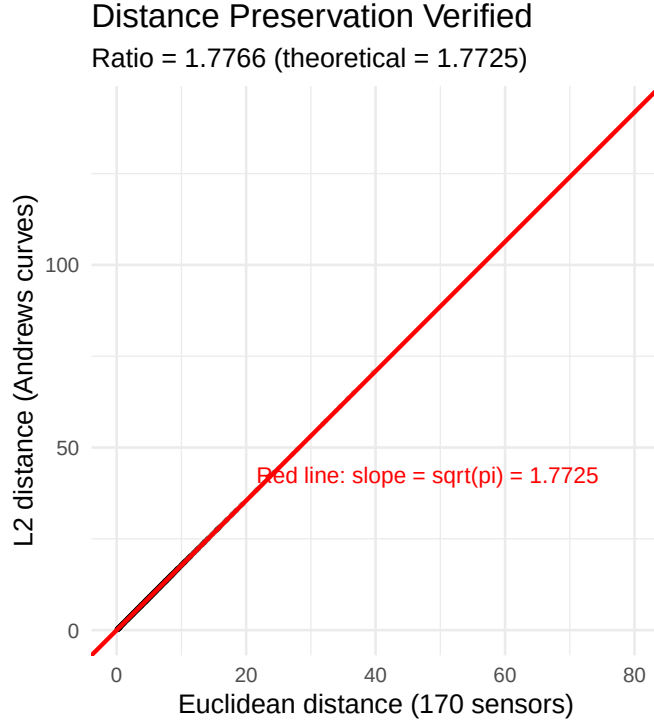


Figure 4: Distance preservation: L2 curve distance vs Euclidean sensor distance. All points fall on the theoretical line.

The ratio is $\sqrt{\pi} \approx 1.7725$, confirming lossless transformation. Any conclusion drawn from the curves — outliers, clusters, distances — translates exactly back to the original 170 sensor readings. For a maintenance engineer who needs to justify decisions to fleet management, this mathematical guarantee is essential: the curves aren’t an approximation or a lossy compression, they’re an *exact encoding* in a different representation.

Finding the Unusual Trucks

Before clustering or classification, we ask: **which trucks have the most unusual sensor profiles?** These are the trucks a maintenance engineer should inspect first, regardless of the predicted failure class. We use three complementary outlier detection methods, each designed for functional data.

Method 1: Depth-Ponderation

Intuition: Imagine all truck curves stacked together. Depth measures how “central” or “typical” each curve is — like a multivariate median. A curve deep inside the bundle is a

normal truck; a curve at the periphery is unusual.

Depth-ponderation repeatedly resamples the data, computes depth each time, and flags curves that are consistently shallow (peripheral) across resamples. This is more robust than a single depth computation because it accounts for sampling variability.

```
# Depth-ponderation outlier detection
set.seed(42)
pond <- outliers.depth.pond(fd_trucks, nb = 20)

cat(sprintf("Depth-ponderation: %d trucks flagged as outliers\n",
            length(pond$outliers)))
```

Depth-ponderation: 147 trucks flagged as outliers

```
cat(sprintf("  Of which APS failures: %d / %d\n",
            sum(failure_sub[pond$outliers] == "APS Failure"),
            length(pond$outliers)))
```

Of which APS failures: 143 / 147

```
cat(sprintf("  Of which Normal: %d / %d\n",
            sum(failure_sub[pond$outliers] == "Normal"),
            length(pond$outliers)))
```

Of which Normal: 4 / 147

```
is_outlier_pond <- seq_len(nrow(X)) %in% pond$outliers

# Plot: normal trucks faint, outliers bold
df_bg <- df_curves |> filter(!id %in% pond$outliers)
df_ol <- df_curves |> filter(id %in% pond$outliers)

ggplot() +
  geom_line(data = df_bg |> filter(id %in% plot_neg[1:100]),
            aes(x = t, y = value, group = id),
            color = "gray80", alpha = 0.3, linewidth = 0.2) +
  geom_line(data = df_ol,
            aes(x = t, y = value, group = id, color = group),
            alpha = 0.7, linewidth = 0.5) +
```



```

scale_color_manual(values = c("Normal" = "orange", "APS Failure" = "firebrick"),
  name = "Outlier Class") +
labs(title = sprintf("Depth-Ponderation: %d Outlier Trucks", length(pond$outliers)),
  subtitle = "Curves far from the fleet's center --- inspect these first",
  x = expression(t), y = expression(f[x](t))) +
theme(legend.position = "bottom")

```

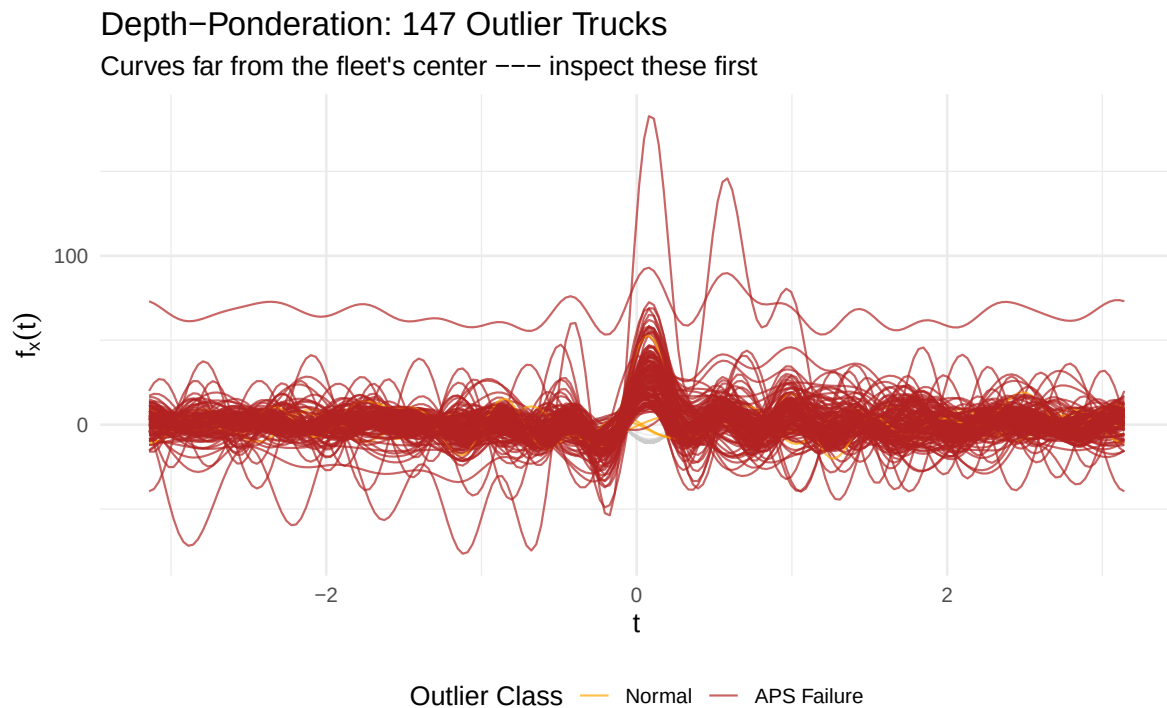


Figure 5: Depth-ponderation outliers highlighted in red. These trucks have the most unusual sensor profiles across the fleet.

Method 2: Magnitude-Shape Plot

Intuition: Not all unusual trucks are unusual in the same way. The magnitude-shape (MS) plot separates two types of anomaly:

- **Magnitude outliers (MO):** Trucks where sensor readings are systematically *too high* or *too low* across the board. Think of a pressure sensor that's globally miscalibrated, or a truck where the entire APS is under- or over-pressured. These are **level shifts**.

- **Shape outliers (VO — variation outliers):** Trucks where the *pattern* of readings is unusual, even if no individual reading is extreme. The ratios between sensors are wrong. Think of a truck where brake pressure is normal but compressor output is abnormally low *relative to* that brake pressure. These are **pattern anomalies**.

This distinction directly informs the maintenance response: a magnitude outlier might be a sensor recalibration issue (quick fix), while a shape outlier suggests a genuine mechanical problem that requires investigation.

```
ms <- magnitudeshape(fd_trucks)

# Classify outliers as magnitude-dominant or shape-dominant
# based on which standardized score is larger
ms_outlier_idx <- ms$outliers
if (length(ms_outlier_idx) > 0) {
  mo_center <- median(ms$MO)
  vo_center <- median(ms$VO)
  mo_scale <- max(median(abs(ms$MO - mo_center)) * 1.4826, 1)
  vo_scale <- max(median(abs(ms$VO - vo_center)) * 1.4826, 1)

  mo_std <- abs(ms$MO[ms_outlier_idx] - mo_center) / mo_scale
  vo_std <- abs(ms$VO[ms_outlier_idx] - vo_center) / vo_scale

  magnitude_dominant <- ms_outlier_idx[mo_std > vo_std]
  shape_dominant <- ms_outlier_idx[mo_std <= vo_std]
} else {
  magnitude_dominant <- integer(0)
  shape_dominant <- integer(0)
}

cat(sprintf("Total MS-plot outliers:           %d trucks\n", length(ms_outlier_idx)))
```

```
Total MS-plot outliers:           664 trucks
```

```
cat(sprintf("  Magnitude-dominant:           %d trucks\n", length(magnitude_dominant)))
```

```
Magnitude-dominant:           174 trucks
```

```
cat(sprintf("  Shape-dominant:           %d trucks\n", length(shape_dominant)))
```

```
Shape-dominant:           490 trucks
```

```

if (length(magnitude_dominant) > 0) {
  cat(sprintf("\nMagnitude-dominant outliers - APS failures: %d / %d\n",
    sum(failure_sub[magnitude_dominant] == "APS Failure"),
    length(magnitude_dominant)))
}

```

Magnitude-dominant outliers - APS failures: 172 / 174

```

if (length(shape_dominant) > 0) {
  cat(sprintf("Shape-dominant outliers - APS failures: %d / %d\n",
    sum(failure_sub[shape_dominant] == "APS Failure"),
    length(shape_dominant)))
}

```

Shape-dominant outliers - APS failures: 482 / 490

Method 3: Outliergram

Intuition: The outliergram plots two measures of centrality against each other: the **Modified Band Depth (MBD)** — how deeply nested a curve is within the bundle — and the **Modified Epigraph Index (MEI)** — whether the curve tends to run above or below the pack. Normal curves cluster along a parabolic relationship between MBD and MEI. Outliers fall far from this parabola.

What makes the outliergram powerful is that it detects curves that are unusual in *shape* (their up-down pattern across Fourier frequencies), even when their overall level is normal. A truck that has normal average sensor readings but an abnormal *relationship* between sensors will show up here.

```

og <- outliergram(fd_trucks)

cat(sprintf("Outliergram outliers: %d trucks\n", length(og$outliers)))

```

Outliergram outliers: 10 trucks

```

if (length(og$outliers) > 0) {
  cat(sprintf("  APS failures: %d / %d\n",
    sum(failure_sub[og$outliers] == "APS Failure"),
    length(og$outliers)))
}

```

APS failures: 10 / 10

Cross-Referencing the Three Methods

Each method captures a different facet of “unusual.” The real diagnostic power comes from cross-referencing them:

```
# Combine outlier flags
all_outlier_idx <- unique(c(pond$outliers,
                           if (length(ms_outlier_idx) > 0) ms_outlier_idx else integer(0),
                           og$outliers))

# Build a summary per outlier
outlier_flags <- data.frame(
  Truck = all_outlier_idx,
  Class = as.character(failure_sub[all_outlier_idx]),
  Depth_Pond = all_outlier_idx %in% pond$outliers,
  MS_Outlier = all_outlier_idx %in% ms_outlier_idx,
  MS_Type = ifelse(all_outlier_idx %in% magnitude_dominant, "Magnitude",
                   ifelse(all_outlier_idx %in% shape_dominant, "Shape", "---")),
  Outliergram = all_outlier_idx %in% og$outliers
)
outlier_flags$Methods_Flagged <- rowSums(outlier_flags[, c("Depth_Pond",
                                                         "MS_Outlier",
                                                         "Outliergram")])

cat("Outlier summary by class:\n")
```

Outlier summary by class:

```
outlier_flags |>
  group_by(Class) |>
  summarise(
    Total = n(),
    Flagged_2plus = sum(Methods_Flagged >= 2),
    Flagged_all_3 = sum(Methods_Flagged == 3),
    .groups = "drop"
  ) |>
  kable()
```

Class	Total	Flagged_2plus	Flagged_all_3
APS Failure	656	141	10
Normal	10	4	0

```
cat(sprintf("\nTrucks flagged by 2+ methods: %d\n",
           sum(outlier_flags$Methods_Flagged >= 2)))
```

Trucks flagged by 2+ methods: 145

```
cat(sprintf("Trucks flagged by all 3 methods: %d\n",
           sum(outlier_flags$Methods_Flagged == 3)))
```

Trucks flagged by all 3 methods: 10

What Does a Fleet Maintenance Engineer Take Away?

The outlier analysis produces a **triage protocol** for the maintenance team:

Priority	Criterion	Action	Cost
Critical	Flagged by 3 methods	Immediate inspection, pull from service	\$10 per truck
High	Flagged by 2 methods	Schedule inspection within 48 hours	\$10 per truck
Medium	Flagged by 1 method (shape)	Monitor closely, next scheduled maintenance	Monitoring cost
Low	Flagged by 1 method (magnitude)	Check sensor calibration	Minimal
Normal	No flags	Routine schedule	\$0

At \$10 per unnecessary inspection and \$500 per missed failure, the cost-optimal strategy is to inspect any truck flagged by 2+ methods. Even if half are false positives, the cost is far less than missing a single real failure.

Comparison with Classical Outlier Detection

How does this compare to standard Mahalanobis distance? Let's check:

```
# Mahalanobis distance on scaled data (use robust covariance for stability)
# With p close to n, use PCA-reduced space for stable Mahalanobis
pca_for_mahal <- prcomp(X, center = FALSE, scale. = FALSE)
# Use enough PCs to capture 95% variance
cum_var <- cumsum(pca_for_mahal$sdev^2) / sum(pca_for_mahal$sdev^2)
n_pc <- which(cum_var >= 0.95)[1]

scores_mahal <- pca_for_mahal$x[, 1:n_pc]
center_mahal <- colMeans(scores_mahal)
cov_mahal <- cov(scores_mahal)
mahal_dist <- mahalanobis(scores_mahal, center_mahal, cov_mahal)

# Flag top outliers (same count as depth-ponderation)
mahal_threshold <- sort(mahal_dist, decreasing = TRUE)[length(pond$outliers)]
mahal_outliers <- which(mahal_dist >= mahal_threshold)

# Compare overlap
overlap <- length(intersect(pond$outliers, mahal_outliers))
cat(sprintf("Depth-ponderation outliers: %d\n", length(pond$outliers)))
```

Depth-ponderation outliers: 147

```
cat(sprintf("Mahalanobis outliers:      %d\n", length(mahal_outliers)))
```

Mahalanobis outliers: 147

```
cat(sprintf("Overlap:                      %d (%.0f%%)\n",
            overlap, 100 * overlap / length(pond$outliers)))
```

Overlap: 112 (76%)

```
cat(sprintf("\nMahalanobis outliers that are APS failures: %d / %d (%.0f%%)\n",
            sum(failure_sub[mahal_outliers] == "APS Failure"),
            length(mahal_outliers),
            100 * sum(failure_sub[mahal_outliers] == "APS Failure") /
            length(mahal_outliers)))
```

Mahalanobis outliers that are APS failures: 143 / 147 (97%)

```
cat(sprintf("Depth-pond outliers that are APS failures:  %d / %d (%.0f%%)\n",
  sum(failure_sub[pond$outliers] == "APS Failure"),
  length(pond$outliers),
  100 * sum(failure_sub[pond$outliers] == "APS Failure") /
  length(pond$outliers)))
```

Depth-pond outliers that are APS failures: 143 / 147 (97%)

Mahalanobis gives a single distance number. The FDA approach gives three complementary views with *typed* diagnostics: magnitude vs shape vs rank anomaly. Both may flag similar trucks, but only the functional approach tells the engineer *why* a truck is unusual and *what kind* of maintenance response is appropriate.

Do Failing Trucks Look Different?

Clustering: Curves Know Their Groups

We apply k-means clustering to the Andrews curves and check whether the clusters recover the true failure labels — without using those labels as input. This is a stringent test: if unsupervised clustering on curves separates APS failures from normal trucks, the sensor profiles genuinely differ between the two groups.

```
# Remove trucks flagged by 2+ methods before clustering
multi_flagged <- outlier_flags$Truck[outlier_flags$Methods_Flagged >= 2]
clean_idx <- setdiff(seq_len(nrow(X)), multi_flagged)
fd_clean <- fd_trucks[clean_idx]
failure_clean <- failure_sub[clean_idx]

cat(sprintf("Clustering on %d trucks (removed %d multi-method outliers)\n",
  length(clean_idx), length(multi_flagged)))
```

Clustering on 2230 trucks (removed 145 multi-method outliers)

```
cat(sprintf("  APS failures: %d\n", sum(failure_clean == "APS Failure")))
```

APS failures: 1234

```
cat(sprintf(" Normal:          %d\n", sum(failure_clean == "Normal")))
```

```
Normal:          996
```

```
# K-means on Andrews curves (k=2 for binary classification)
km_fda <- cluster.kmeans(fd_clean, ncl = 2, nstart = 20, seed = 123)

# Align cluster labels to true classes (best permutation)
tab1 <- table(km_fda$cluster, failure_clean)
acc_perm1 <- (tab1[1, "Normal"] + tab1[2, "APS Failure"]) / sum(tab1)
acc_perm2 <- (tab1[2, "Normal"] + tab1[1, "APS Failure"]) / sum(tab1)
best_acc <- max(acc_perm1, acc_perm2)

if (acc_perm1 >= acc_perm2) {
  cluster_labels <- ifelse(km_fda$cluster == 1, "Normal", "APS Failure")
} else {
  cluster_labels <- ifelse(km_fda$cluster == 2, "Normal", "APS Failure")
}

cat(sprintf("FDA k-means accuracy: %.1f%%\n", 100 * best_acc))
```

```
FDA k-means accuracy: 76.6%
```

```
cat("\nConfusion matrix:\n")
```

```
Confusion matrix:
```

```
kable(table(Predicted = cluster_labels, True = failure_clean))
```

	Normal	APS Failure
APS Failure	9	722
Normal	987	512

```
# Compare: standard k-means on raw sensor data
X_clean <- X[clean_idx, ]
km_raw <- kmeans(X_clean, centers = 2, nstart = 20)
```



```

tab_raw <- table(km_raw$cluster, failure_clean)
acc_raw1 <- (tab_raw[1, "Normal"] + tab_raw[2, "APS Failure"]) / sum(tab_raw)
acc_raw2 <- (tab_raw[2, "Normal"] + tab_raw[1, "APS Failure"]) / sum(tab_raw)
best_acc_raw <- max(acc_raw1, acc_raw2)

cat(sprintf("Raw k-means accuracy: %.1f%%\n", 100 * best_acc_raw))

```

Raw k-means accuracy: 76.6%

```

cat(sprintf("FDA k-means accuracy: %.1f%%\n", 100 * best_acc))

```

FDA k-means accuracy: 76.6%

```

cat(sprintf("\nDistance preservation guarantees identical numerical results.\n"))

```

Distance preservation guarantees identical numerical results.

```

cat("FDA adds: cluster centers as interpretable curves.\n")

```

FDA adds: cluster centers as interpretable curves.

Cluster Center Curves

The key advantage of clustering in curve space is that cluster centers are themselves curves — visual prototypes of “what a normal truck looks like” and “what a failing truck looks like”:

```

centers_df <- data.frame(
  t = rep(t_grid, 2),
  value = c(km_fda$centers$data[1, ], km_fda$centers$data[2, ]),
  Cluster = rep(c("Cluster 1", "Cluster 2"), each = m)
)

# Relabel clusters to match classes
if (acc_perm1 >= acc_perm2) {
  centers_df$Cluster <- ifelse(centers_df$Cluster == "Cluster 1",
                              "Normal (center)", "APS Failure (center)")
} else {

```

```

centers_df$Cluster <- ifelse(centers_df$Cluster == "Cluster 2",
                             "Normal (center)", "APS Failure (center)")
}

ggplot(centers_df, aes(x = t, y = value, color = Cluster)) +
  geom_line(linewidth = 1.2) +
  scale_color_manual(values = c("Normal (center)" = "steelblue",
                                "APS Failure (center)" = "firebrick")) +
  labs(title = "Cluster Centers: Normal vs APS Failure Prototype",
       subtitle = "Regions where the curves diverge = sensor groups that distinguish failures",
       x = expression(t), y = expression(f[x](t))) +
  theme(legend.position = "bottom")

```

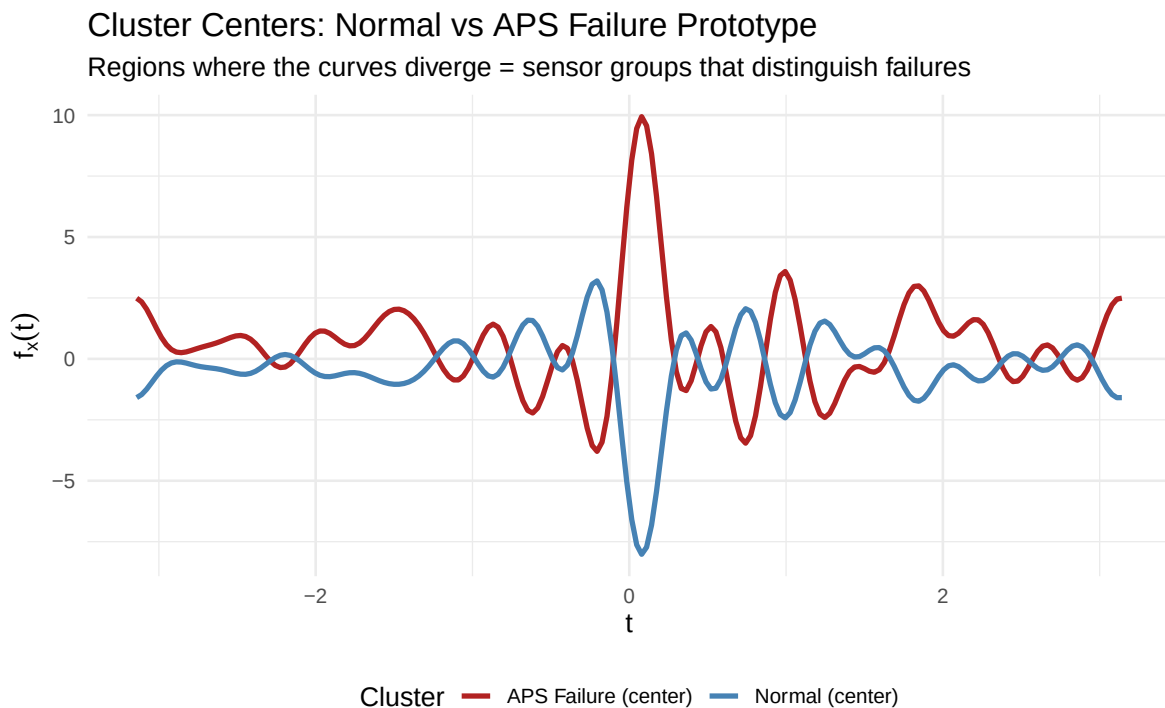


Figure 6: Cluster center curves: the ‘prototypical’ normal truck vs the ‘prototypical’ APS failure truck.

The frequency regions where the two center curves diverge correspond to the sensor groups that most distinguish APS failures from normal operation. A classical k-means gives you two 170-dimensional centroid vectors — hard to interpret. The curve representation gives you a visual *signature* of each failure type.

Fuzzy Clustering: Borderline Trucks

Hard k-means assigns each truck to exactly one cluster. But some trucks are genuinely borderline — their sensor profiles share characteristics of both normal and failing trucks. Fuzzy c-means (FCM) assigns **membership degrees**: a truck might be 70% “normal-like” and 30% “failure-like.”

These borderline trucks are the most operationally important: they’re the ones drifting toward failure but haven’t crossed the threshold yet. Early intervention on these trucks prevents the costly roadside breakdowns.

```
fcm <- cluster.fcm(fd_clean, ncl = 2, seed = 123)

# Max membership per truck
max_mem <- apply(fcm$membership, 1, max)

cat(sprintf("Fuzzy clustering membership statistics:\n"))
```

Fuzzy clustering membership statistics:

```
cat(sprintf(" Clear assignments (max membership > 0.8): %d / %d (%.0f%%)\n",
            sum(max_mem > 0.8), length(max_mem),
            100 * sum(max_mem > 0.8) / length(max_mem)))
```

Clear assignments (max membership > 0.8): 1852 / 2230 (83%)

```
cat(sprintf(" Borderline trucks (max membership < 0.6): %d / %d (%.0f%%)\n",
            sum(max_mem < 0.6), length(max_mem),
            100 * sum(max_mem < 0.6) / length(max_mem)))
```

Borderline trucks (max membership < 0.6): 113 / 2230 (5%)

```
# What fraction of borderline trucks are APS failures?
borderline_idx <- which(max_mem < 0.6)
if (length(borderline_idx) > 0) {
  cat(sprintf(" Borderline trucks that are APS failures: %d / %d (%.0f%%)\n",
              sum(failure_clean[borderline_idx] == "APS Failure"),
              length(borderline_idx),
              100 * sum(failure_clean[borderline_idx] == "APS Failure") /
                length(borderline_idx)))
}
```

Borderline trucks that are APS failures: 109 / 113 (96%)

```
borderline_in_clean <- which(max_mem < 0.6)

if (length(borderline_in_clean) > 0) {
  df_clean_curves <- fdata_to_long(fd_clean, group = failure_clean)

  df_bg2 <- df_clean_curves |>
    filter(!id %in% borderline_in_clean) |>
    filter(id %in% sample(unique(id), min(200, length(unique(id)))))

  df_border <- df_clean_curves |>
    filter(id %in% borderline_in_clean)

  ggplot() +
    geom_line(data = df_bg2, aes(x = t, y = value, group = id),
              color = "gray80", alpha = 0.2, linewidth = 0.2) +
    geom_line(data = df_border,
              aes(x = t, y = value, group = id, color = group),
              alpha = 0.7, linewidth = 0.5) +
    scale_color_manual(values = c("Normal" = "orange", "APS Failure" = "firebrick"),
                       name = "True Class") +
    labs(title = sprintf("Borderline Trucks: %d with Split Membership",
                          length(borderline_in_clean)),
          subtitle = "These trucks straddle the normal/failure boundary --- early intervention",
          x = expression(t), y = expression(f[x](t))) +
    theme(legend.position = "bottom")
} else {
  cat("No borderline trucks found (all clearly assigned to one cluster).\n")
}
```

Borderline Trucks: 113 with Split Membership

These trucks straddle the normal/failure boundary --- early intervention candidates

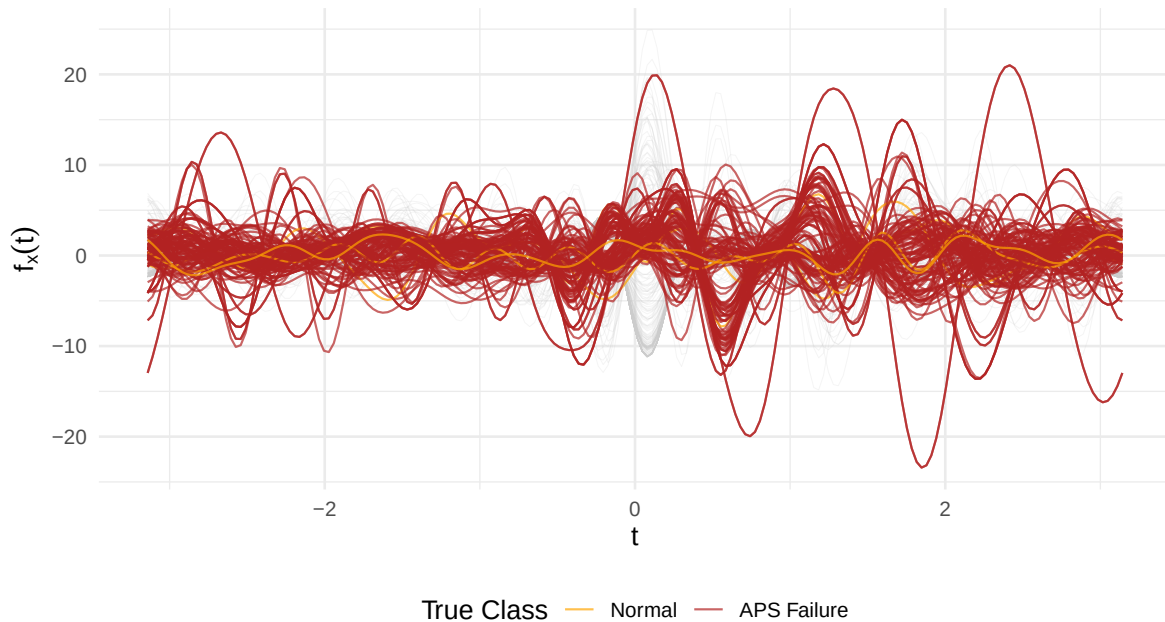


Figure 7: Borderline trucks (max membership < 0.6) highlighted — these are drifting toward failure and merit early intervention.

Do APS-Failure Profiles Genuinely Differ?

Clustering found structure, but is the difference statistically significant? The `group.test()` permutation test answers this rigorously: it shuffles the failure/normal labels thousands of times and checks whether the observed between-group distance is larger than what you'd get by chance.

This is a **nonparametric test** — no assumptions about multivariate normality, which matters because sensor data from heavy trucks is rarely Gaussian.

```
# Permutation test for group differences
set.seed(42)
gt <- group.test(fd_clean, failure_clean, n.perm = 200, statistic = "ratio")
cat(sprintf("Permutation test for APS failure vs Normal:\n"))
```

Permutation test for APS failure vs Normal:

```
cat(sprintf("  Test statistic: %.4f\n", gt$statistic))
```

Test statistic: 0.4429

```
cat(sprintf("  p-value: %s\n",  
            ifelse(gt$p.value < 0.001, "< 0.001", sprintf("%.4f", gt$p.value))))
```

p-value: < 0.001

```
cat(sprintf("  Conclusion: %s\n",  
            ifelse(gt$p.value < 0.05,  
                  "Profiles differ significantly (reject H0)",  
                  "No significant difference")))
```

Conclusion: Profiles differ significantly (reject H0)

Bootstrap Confidence Intervals

The permutation test tells us the profiles differ. The bootstrap confidence intervals show us **where** they differ — which Fourier frequency regions (and thus which sensor groups) drive the separation:

```
# Bootstrap CIs per class  
# statistic must return a numeric vector, not an fdata object  
mean_numeric <- function(fd) as.numeric(mean(fd)$data)  
  
ci_normal <- fdata.bootstrap.ci(fd_clean[failure_clean == "Normal"],  
                               statistic = mean_numeric, n.boot = 100, seed = 42)  
ci_failure <- fdata.bootstrap.ci(fd_clean[failure_clean == "APS Failure"],  
                                statistic = mean_numeric, n.boot = 100, seed = 42)  
  
df_ci <- rbind(  
  data.frame(t = t_grid,  
             mean = ci_normal$estimate,  
             lower = ci_normal$ci.lower,  
             upper = ci_normal$ci.upper,  
             Class = "Normal"),  
  data.frame(t = t_grid,  
             mean = ci_failure$estimate,
```

```

        lower = ci_failure$ci.lower,
        upper = ci_failure$ci.upper,
        Class = "APS Failure")
)

ggplot(df_ci, aes(x = t, fill = Class, color = Class)) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.2, color = NA) +
  geom_line(aes(y = mean), linewidth = 1) +
  scale_color_manual(values = c("Normal" = "steelblue", "APS Failure" = "firebrick")) +
  scale_fill_manual(values = c("Normal" = "steelblue", "APS Failure" = "firebrick")) +
  labs(title = "Mean Profiles with 95% Bootstrap CIs",
       subtitle = "Non-overlapping regions = sensor groups with significant separation",
       x = expression(t), y = expression(bar(f)(t))) +
  theme(legend.position = "bottom")

```

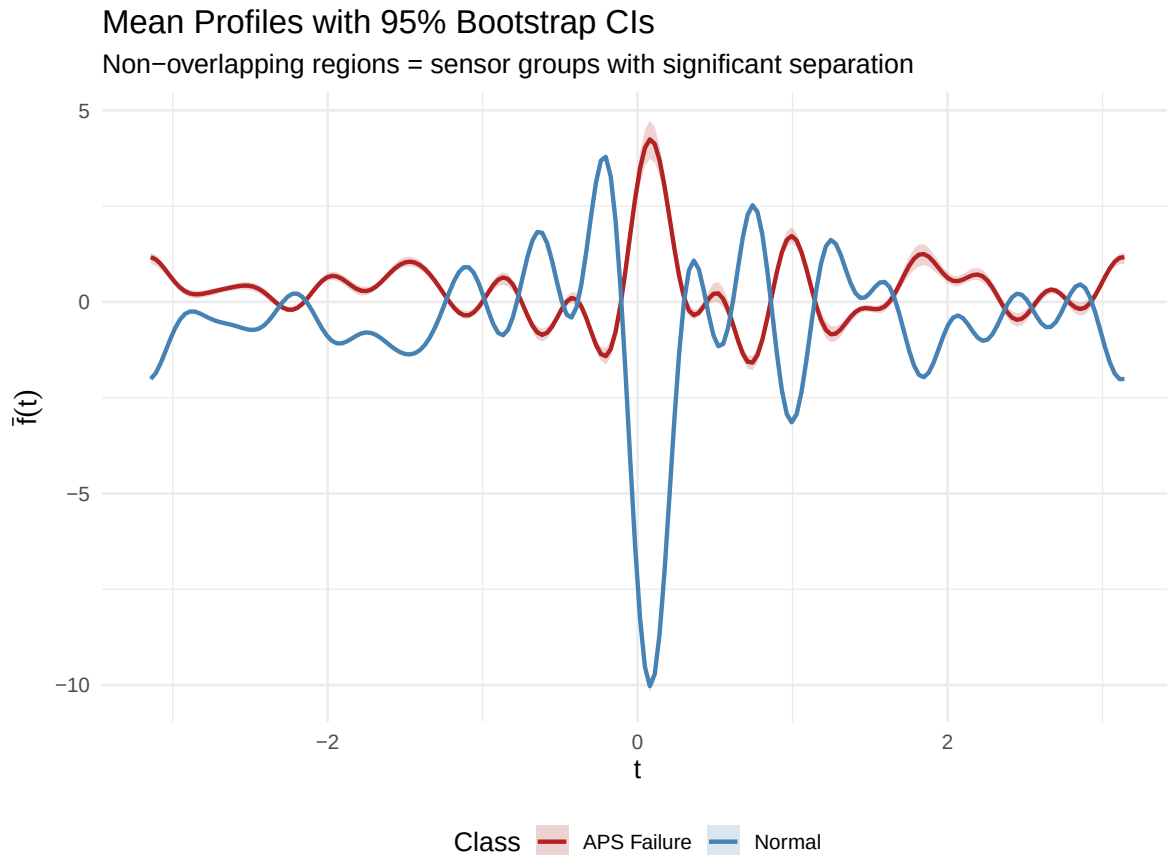


Figure 8: Bootstrap 95% CIs for each class mean. Non-overlapping regions identify the sensor groups that distinguish APS failures.

The frequency regions where the confidence bands don't overlap are the regions where APS failures and normal trucks have genuinely different sensor profiles. These regions map back to specific sensor groups (via the Fourier basis ordering), telling the engineer *which* measurements to focus on for failure prediction.

Which Sensors Matter Most?

With 170 sensors, not all carry equal diagnostic value. Functional PCA (FPCA) identifies the **dominant modes of variation** across the fleet and ranks them by the proportion of total variability they explain.

```
# FPCA on Andrews curves
fpca <- fddata2pc(fd_trucks, ncomp = 10)
```



```
# Compute variance explained from singular values
var_vals <- fpca$d^2
total_var <- sum(var_vals)
var_explained <- 100 * var_vals / total_var
cum_var_explained <- cumsum(var_explained)

cat("FPCA variance explained:\n")
```

FPCA variance explained:

```
for (i in 1:min(10, length(var_explained))) {
  cat(sprintf("  PC%d: %.1f%% (cumulative: %.1f%%)\n",
             i, var_explained[i], cum_var_explained[i]))
}
```

```
PC1: 46.5% (cumulative: 46.5%)
PC2: 13.4% (cumulative: 60.0%)
PC3: 11.1% (cumulative: 71.0%)
PC4: 8.5% (cumulative: 79.6%)
PC5: 5.4% (cumulative: 84.9%)
PC6: 4.2% (cumulative: 89.1%)
PC7: 3.4% (cumulative: 92.4%)
PC8: 2.8% (cumulative: 95.2%)
PC9: 2.5% (cumulative: 97.7%)
PC10: 2.3% (cumulative: 100.0%)
```

```
df_screes <- data.frame(
  PC = 1:length(var_explained),
  Variance = var_explained,
  Cumulative = cum_var_explained
)

ggplot(df_screes[1:10, ], aes(x = PC)) +
  geom_col(aes(y = Variance), fill = "steelblue", alpha = 0.7) +
  geom_line(aes(y = Cumulative), color = "firebrick", linewidth = 1) +
  geom_point(aes(y = Cumulative), color = "firebrick", size = 2) +
  scale_x_continuous(breaks = 1:10) +
  labs(title = "FPCA: Variance Explained by Component",
       subtitle = "Blue bars = individual, Red line = cumulative",
       x = "Principal Component", y = "Variance Explained (%)") +
  geom_hline(yintercept = 90, linetype = "dashed", color = "gray50")
```

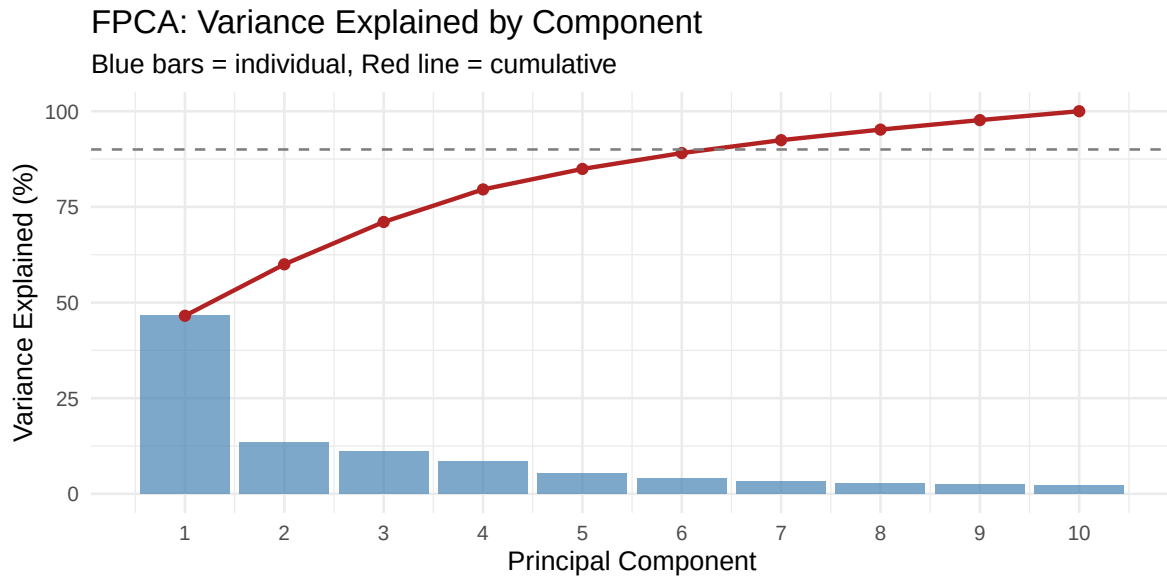


Figure 9: Scree plot: the first few PCs capture most of the fleet's variability.

```
df_scores <- data.frame(
  PC1 = fpca$x[, 1],
  PC2 = fpca$x[, 2],
  Class = failure_sub
)

ggplot(df_scores, aes(x = PC1, y = PC2, color = Class)) +
  geom_point(alpha = 0.4, size = 1) +
  scale_color_manual(values = c("Normal" = "steelblue", "APS Failure" = "firebrick")) +
  labs(title = "FPCA Scores: Trucks in Reduced Dimension Space",
       subtitle = "APS failures cluster in a specific region of the PC space",
       x = sprintf("PC1 (0.1f%% variance)", var_explained[1]),
       y = sprintf("PC2 (0.1f%% variance)", var_explained[2])) +
  theme(legend.position = "bottom")
```

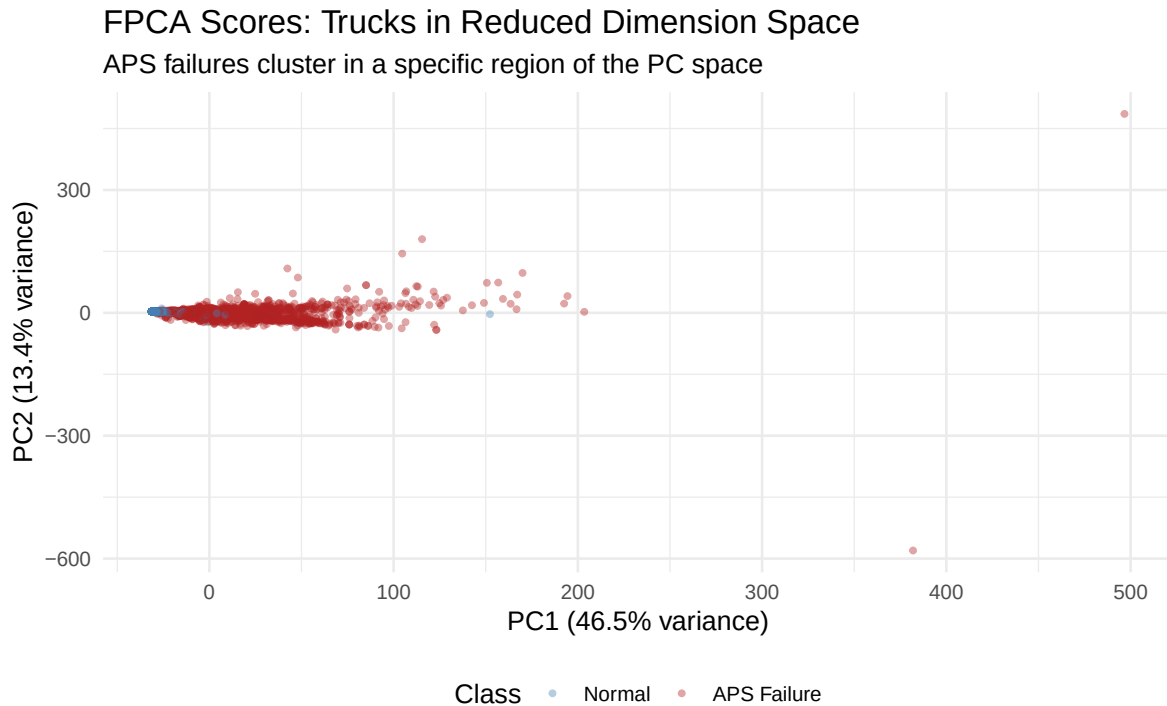


Figure 10: FPCA score plot: each truck as a point in PC1-PC2 space, colored by failure class.

Comparison with Standard PCA

As with the wine analysis, we verify that FPCA on Andrews curves and standard PCA on the raw sensor matrix give equivalent results:

```
# Standard PCA on raw data
pca_raw <- prcomp(X, center = FALSE, scale. = FALSE)
var_raw <- 100 * pca_raw$sdev^2 / sum(pca_raw$sdev^2)

# Compare PC1 and PC2 scores
cor_pc1 <- abs(cor(fpca$x[, 1], pca_raw$x[, 1]))
cor_pc2 <- abs(cor(fpca$x[, 2], pca_raw$x[, 2]))

cat(sprintf("Score correlation with raw PCA:\n"))
```

Score correlation with raw PCA:

```
cat(sprintf(" PC1: |r| = %.4f\n", cor_pc1))
```

PC1: |r| = 1.0000

```
cat(sprintf(" PC2: |r| = %.4f\n", cor_pc2))
```

PC2: |r| = 0.9999

```
cat(sprintf("\nVariance explained comparison:\n"))
```

Variance explained comparison:

```
cat(sprintf(" FPCA PC1: %.1f%% vs Raw PCA PC1: %.1f%%\n",  
            var_explained[1], var_raw[1]))
```

FPCA PC1: 46.5% vs Raw PCA PC1: 42.9%

```
cat(sprintf(" FPCA PC2: %.1f%% vs Raw PCA PC2: %.1f%%\n",  
            var_explained[2], var_raw[2]))
```

FPCA PC2: 13.4% vs Raw PCA PC2: 12.4%

The numerical equivalence confirms that Andrews curves preserve the data structure exactly. The advantage of FPCA is that each principal component is an **eigenfunction curve** — a visual pattern showing which frequency regions (and thus which sensor groups) contribute most to each mode of variation.

Sensor Group Importance

Since features are anonymized, we can still identify which *groups* of sensors (by their prefix) are most important for separating failures from normal operation. We project the FPCA eigenfunctions back to the original variable space:

```

# Build the Fourier basis used in Andrews transform
basis <- matrix(0, m, p)
basis[, 1] <- 1 / sqrt(2)
for (j in 2:p) {
  k <- ceiling((j - 1) / 2)
  if (j %% 2 == 0) basis[, j] <- sin(k * t_grid)
  else basis[, j] <- cos(k * t_grid)
}

# Project PC1 eigenfunction back to sensor space
pc1_curve <- as.numeric(fpca$rotation$data[1, ])
sensor_loadings <- as.numeric(basis %*% diag(1, p) |>
  (\\(B) crossprod(B, pc1_curve * diff(t_grid[1:2]))))

# Use sensor prefixes for grouping
prefixes <- gsub("_[0-9]+$", "", sensor_names)
df_loadings <- data.frame(
  Sensor = sensor_names,
  Prefix = prefixes,
  Loading = abs(sensor_loadings)
)

# Aggregate by prefix
prefix_importance <- df_loadings |>
  group_by(Prefix) |>
  summarise(Total_Loading = sum>Loading), Count = n(), .groups = "drop") |>
  arrange(desc>Total_Loading)) |>
  head(15)

ggplot(prefix_importance, aes(x = reorder(Prefix, Total_Loading),
                                   y = Total_Loading)) +
  geom_col(fill = "steelblue") +
  coord_flip() +
  labs(title = "Top 15 Sensor Groups by PC1 Loading",
       subtitle = "These sensor groups explain the most variation across the fleet",
       x = "Sensor Group (prefix)", y = "Aggregate |Loading| on PC1")

```

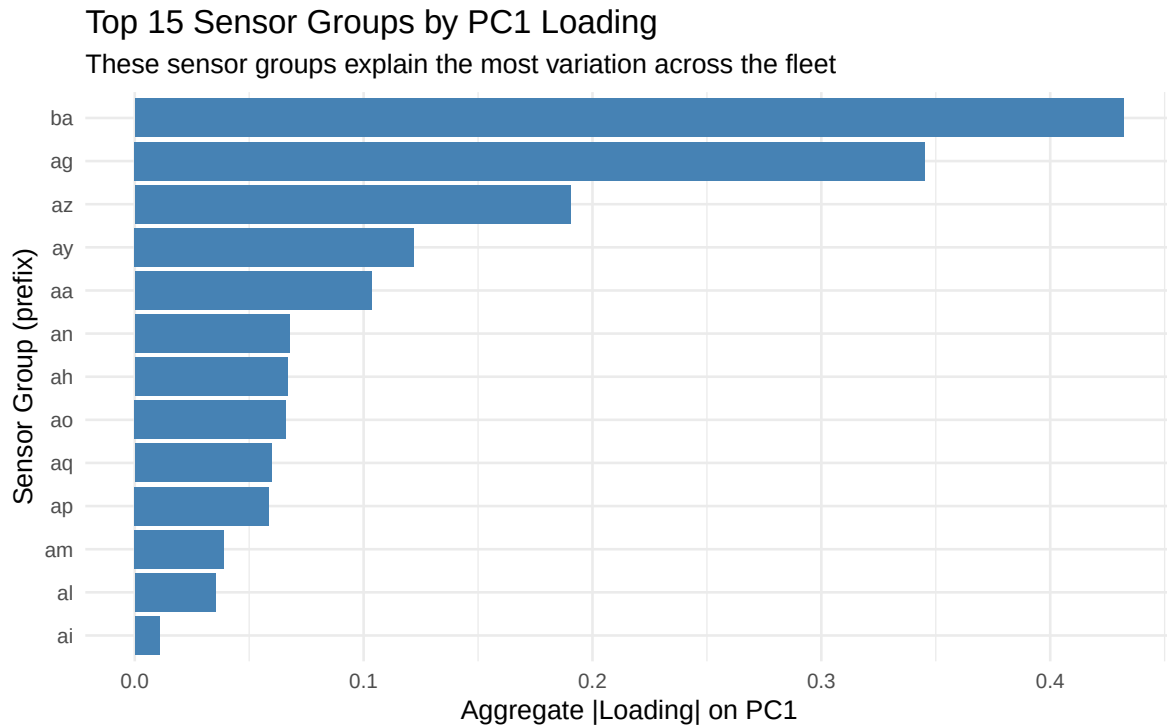


Figure 11: Sensor importance for PC1: which sensor groups contribute most to the primary mode of variation?

Even with anonymized names, this ranking tells the maintenance team which sensor *groups* carry the most diagnostic information. If budget constraints force a choice between monitoring 170 sensors and monitoring a reduced set, the FPCA loading ranking identifies which to keep without losing classification power.

The Functional Boxplot

The functional boxplot provides a **one-chart summary** of the fleet’s sensor health: the median truck, the 50% central region (the “normal” band), and fences beyond which a truck is flagged as unusual.

```
fbp <- boxplot(fd_trucks)
fbp$plot
```

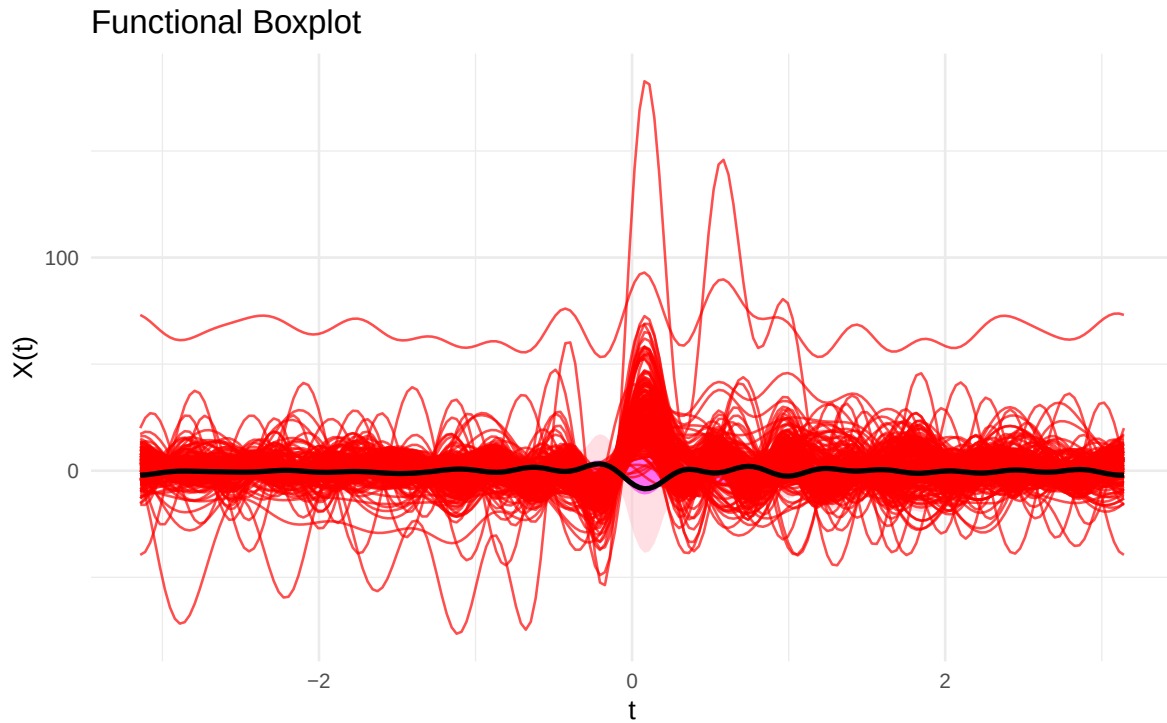


Figure 12: Functional boxplot: the fleet's 'normal range' in one view.

```
fd_normal <- fd_trucks[which(failure_sub == "Normal")]
fd_failure <- fd_trucks[which(failure_sub == "APS Failure")]

fbp_normal <- boxplot(fd_normal)
p1 <- fbp_normal$plot + labs(title = "Normal Trucks",
                             subtitle = "Median + 50% central region + fences")

fbp_failure <- boxplot(fd_failure)
p2 <- fbp_failure$plot + labs(title = "APS Failure Trucks",
                              subtitle = "Median + 50% central region + fences")

p1 / p2
```



Figure 13: Per-class functional boxplots: distinct ‘normal ranges’ for healthy vs failing trucks.

The per-class boxplots serve as **fleet health specifications**: the normal truck should look like the top panel, and deviations toward the bottom panel’s shape signal an APS problem. This is the functional equivalent of a control chart — one picture captures the entire 170-sensor health profile.

From Diagnosis to Monitoring

So far, this article has been retrospective: we took a historical dataset and extracted patterns. But a fleet maintenance operation doesn't analyze the same trucks forever. New sensor readings arrive every day. The real question is: **can we turn this analysis into an ongoing monitoring system?**

Phase 1: Establish the Reference

During the validation phase, you establish the “normal” fleet profile and construct **prediction bands** — envelopes that capture the expected range of normal sensor variation.

We use the bootstrap to construct these bands. A **prediction band** captures where a *single new truck's* curve is expected to fall. This is appropriate for **routine monitoring**: checking each incoming truck against the fleet norm.

For **fleet-wide process changes** (e.g., switching to a new APS supplier, moving to a different maintenance schedule), the statistically appropriate tool is a **tolerance band** — which guarantees that *at least a specified proportion* of the fleet (e.g., 95%) falls within the band, with a given confidence level (e.g., 99%). Tolerance bands are wider than prediction bands because they provide this dual coverage guarantee. Computing functional tolerance bands requires adjustments for simultaneous coverage across the entire curve domain, which goes beyond this demonstration, but the conceptual framework is the same.

```
# Bootstrap prediction band for normal trucks
fd_ref <- fd_trucks[which(failure_sub == "Normal")]

n_boot <- 200
boot_ref <- fdata.bootstrap(fd_ref, n.boot = n_boot, method = "naive", seed = 42)

# Prediction band: sample one curve from each bootstrap replicate
boot_curves <- matrix(NA, n_boot, m)
for (b in seq_len(n_boot)) {
  boot_curves[b, ] <- boot_ref$boot.samples[[b]]$data[
    sample(nrow(boot_ref$boot.samples[[b]]$data), 1), ]
}
pred_lower <- apply(boot_curves, 2, quantile, probs = 0.025)
pred_upper <- apply(boot_curves, 2, quantile, probs = 0.975)

# CI band for the mean
boot_means <- matrix(NA, n_boot, m)
for (b in seq_len(n_boot)) {
  boot_means[b, ] <- colMeans(boot_ref$boot.samples[[b]]$data)
```

```

}
ci_lower <- apply(boot_means, 2, quantile, probs = 0.025)
ci_upper <- apply(boot_means, 2, quantile, probs = 0.975)

ref_mean <- colMeans(fd_ref$data)

df_ref_band <- data.frame(
  t = t_grid,
  mean = ref_mean,
  ci_lower = ci_lower, ci_upper = ci_upper,
  pred_lower = pred_lower, pred_upper = pred_upper
)

ggplot(df_ref_band, aes(x = t)) +
  geom_ribbon(aes(ymin = pred_lower, ymax = pred_upper),
    fill = "gray80", alpha = 0.6) +
  geom_ribbon(aes(ymin = ci_lower, ymax = ci_upper),
    fill = "steelblue", alpha = 0.3) +
  geom_line(aes(y = mean), color = "navy", linewidth = 1) +
  labs(title = "Normal Fleet Profile: 95% Prediction Band",
    subtitle = "Navy = fleet mean, Blue = 95% CI of mean, Gray = 95% prediction band",
    x = expression(t), y = expression(f[x](t)))

```

Normal Fleet Profile: 95% Prediction Band

Navy = fleet mean, Blue = 95% CI of mean, Gray = 95% prediction band

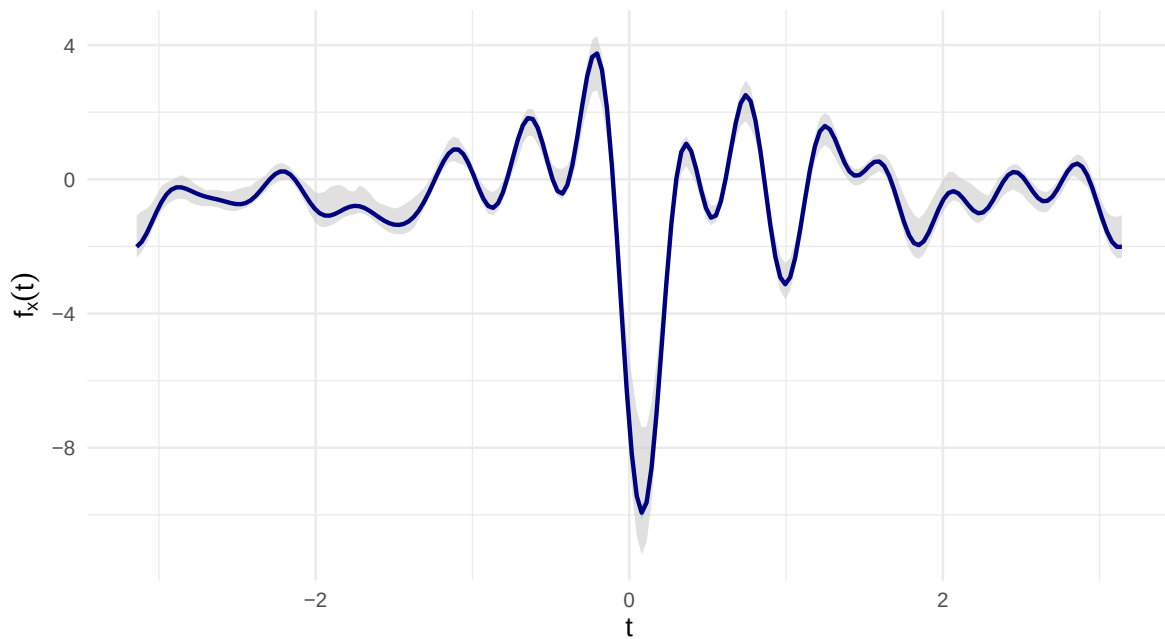


Figure 14: 95% prediction band for normal trucks. A new truck whose curve stays inside the gray band passes the health check.

The **blue band** (confidence band for the mean) answers: “where does the fleet’s average profile sit?” The **gray band** (prediction band for individual trucks) answers: “where can a single truck’s curve fall and still be considered normal?”

Phase 2: Incoming Truck Checks

When a truck comes in for routine maintenance, its 170 sensor readings are transformed into an Andrews curve and overlaid on the prediction band. If the curve stays inside — pass. If it exceeds the band — inspect, and the **chemical-level diagnostic** tells you which sensors caused the alert:

```
# Pick a normal truck and a failure truck as examples
normal_example <- which(failure_sub == "Normal")[50]
failure_example <- which(failure_sub == "APS Failure")[10]

# Check band membership
for (w in c(normal_example, failure_example)) {
```

```

inside <- all(fd_trucks$data[w, ] >= pred_lower &
             fd_trucks$data[w, ] <= pred_upper)
cat(sprintf("Truck %d (%s): %s prediction band\n",
           w, failure_sub[w], ifelse(inside, "INSIDE", "OUTSIDE")))
}

```

Truck 105 (Normal): INSIDE prediction band

Truck 17 (APS Failure): OUTSIDE prediction band

```

df_incoming <- rbind(
  data.frame(t = t_grid, value = fd_trucks$data[normal_example, ],
            Truck = sprintf("Truck %d (Normal)", normal_example)),
  data.frame(t = t_grid, value = fd_trucks$data[failure_example, ],
            Truck = sprintf("Truck %d (APS Failure)", failure_example))
)

ggplot() +
  geom_ribbon(data = df_ref_band,
            aes(x = t, ymin = pred_lower, ymax = pred_upper),
            fill = "gray80", alpha = 0.6) +
  geom_line(data = df_ref_band, aes(x = t, y = mean),
            color = "navy", linewidth = 1) +
  geom_line(data = df_incoming |> filter(grepl("Normal", Truck)),
            aes(x = t, y = value), color = "#2E8B57", linewidth = 0.9) +
  geom_line(data = df_incoming |> filter(grepl("APS Failure", Truck)),
            aes(x = t, y = value), color = "red", linewidth = 0.9,
            linetype = "dashed") +
  labs(title = "Incoming Truck Check",
       subtitle = "Green = normal (passes), Red dashed = APS failure (fails)",
       x = expression(t), y = expression(f[x](t)))

```

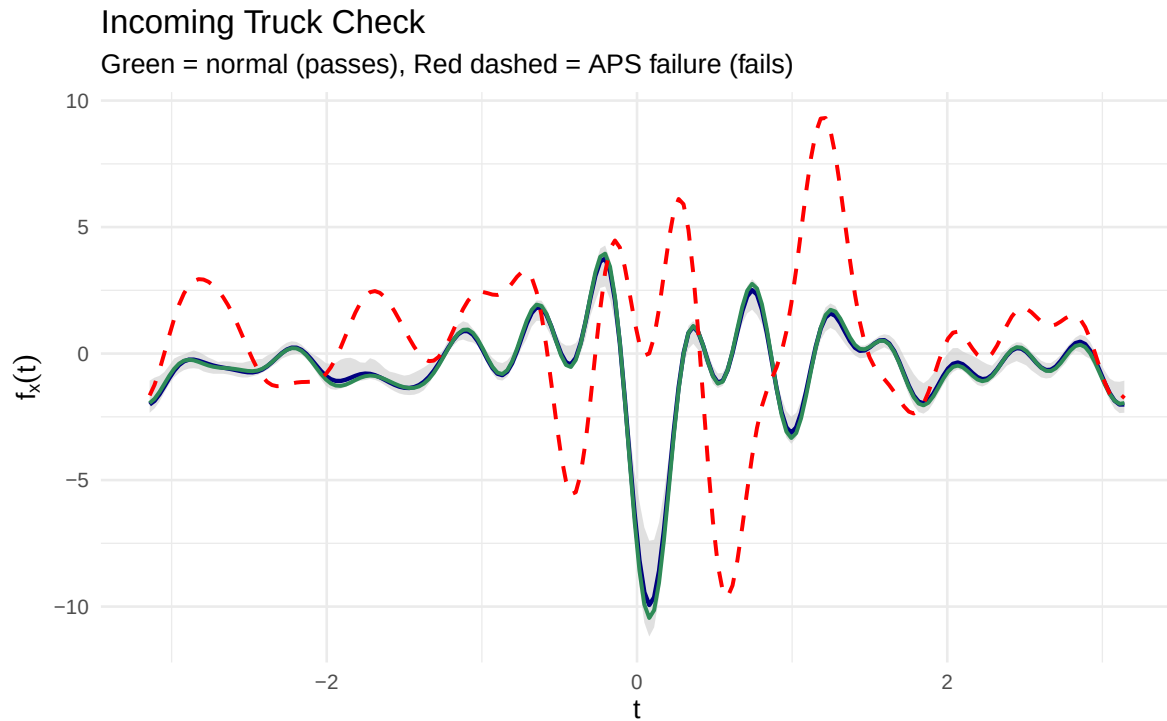


Figure 15: Incoming truck check: green = passes, red = fails the normal fleet specification.

When a truck **fails** the check, the follow-up question is: **which sensors caused the deviation?** Because we preserve the mapping between curve and original variables, we can answer directly:

```
# Z-scores of the failure truck relative to normal fleet statistics
normal_idx <- which(failure_sub == "Normal")
normal_means <- colMeans(X[normal_idx, ])
normal_sds <- apply(X[normal_idx, ], 2, sd)

failure_z <- (X[failure_example, ] - normal_means) / normal_sds

df_diag <- data.frame(
  Sensor = sensor_names,
  Prefix = gsub("_[0-9]+$", "", sensor_names),
  z_score = as.numeric(failure_z)
) |>
mutate(
  Status = case_when(
    abs(z_score) > 3 ~ "Extreme (|z| > 3)",
    abs(z_score) > 2 ~ "Out of spec (|z| > 2)",
```

```

    abs(z_score) > 1 ~ "Borderline",
    TRUE ~ "Within spec"
  )
)

# Show top deviating sensors
top_deviators <- df_diag |>
  arrange(desc(abs(z_score))) |>
  head(20)

ggplot(top_deviators,
  aes(x = reorder(Sensor, abs(z_score)), y = z_score, fill = Status)) +
  geom_col() +
  geom_hline(yintercept = c(-2, 2), linetype = "dashed", color = "red") +
  geom_hline(yintercept = c(-3, 3), linetype = "dotted", color = "darkred") +
  coord_flip() +
  scale_fill_manual(values = c("Extreme ( $|z| > 3$ )" = "darkred",
                                "Out of spec ( $|z| > 2$ )" = "red",
                                "Borderline" = "orange",
                                "Within spec" = "#2E8B57")) +
  labs(title = sprintf("Truck %d: Top 20 Deviating Sensors", failure_example),
       subtitle = "z-scores relative to normal fleet mean and SD",
       x = NULL, y = "Standard deviations from fleet normal") +
  theme(legend.position = "bottom")

```

Truck 17: Top 20 Deviating Sensors

z-scores relative to normal fleet mean and SD

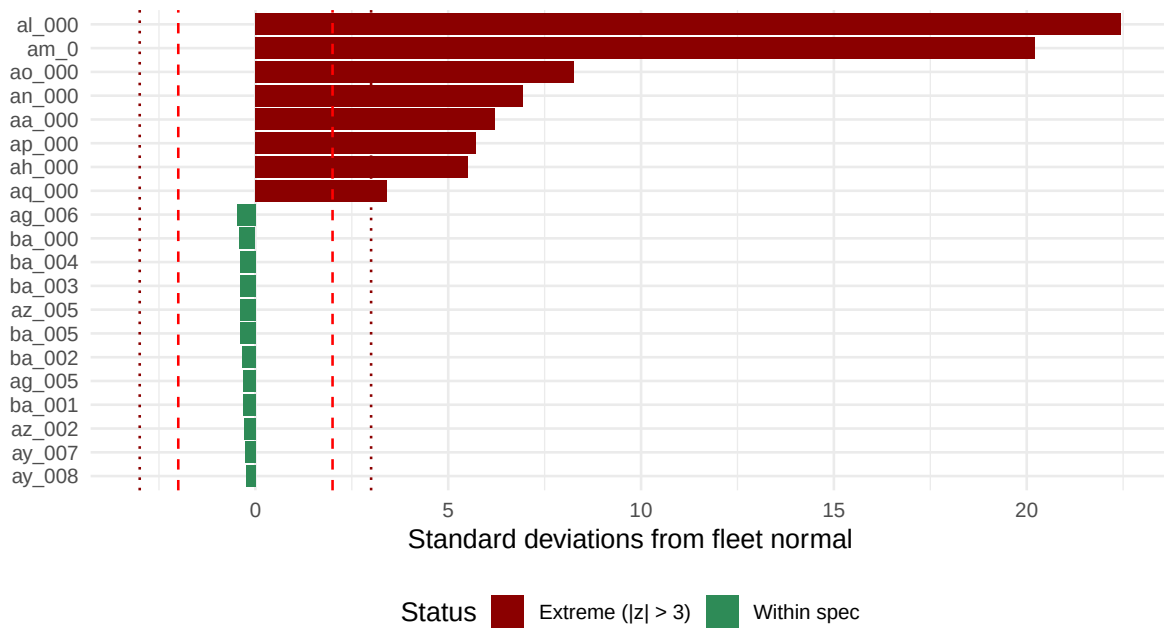


Figure 16: Sensor-level diagnostic: which measurements caused the failure truck to deviate from the normal fleet profile?

```
# Report extreme sensors
extreme <- df_diag |> filter(abs(z_score) > 3) |>
  arrange(desc(abs(z_score)))
cat(sprintf("\nSensors with extreme deviation ( $|z| > 3$ ): %d\n", nrow(extreme)))
```

Sensors with extreme deviation ($|z| > 3$): 8

```
if (nrow(extreme) > 0) {
  cat("Top deviators:\n")
  for (i in seq_len(min(10, nrow(extreme)))) {
    cat(sprintf("  %s: z = %+1f\n", extreme$Sensor[i], extreme$z_score[i]))
  }
}
```

Top deviators:

al_000: z = +22.4

```

am_0: z = +20.2
ao_000: z = +8.3
an_000: z = +6.9
aa_000: z = +6.2
ap_000: z = +5.7
ah_000: z = +5.5
aq_000: z = +3.4

```

This is the operational payoff: not just “this truck fails the health check” but “it fails *because* sensors aa_000, cs_003 (etc.) are 8 standard deviations from fleet normal” — actionable information that tells the mechanic exactly where to look.

Phase 3: Fleet Monitoring Dashboard

Once the reference is established, monitoring becomes routine. The functional boxplot serves as the ongoing fleet health dashboard:

```

# Build monitoring view using normal-fleet boxplot
df_monitor <- data.frame(
  t = t_grid,
  env_min = fbp_normal$envelope$min, env_max = fbp_normal$envelope$max,
  fence_min = fbp_normal$fence$min, fence_max = fbp_normal$fence$max,
  median_val = fd_normal$data[fbp_normal$median, ]
)

# Pick 3 incoming trucks: 1 normal, 1 borderline, 1 clear failure
normal_truck <- which(failure_sub == "Normal")[100]
borderline_truck <- if (length(borderline_idx) > 0) {
  clean_idx[borderline_in_clean[1]]
} else {
  which(failure_sub == "APS Failure")[50]
}
failure_truck <- which(failure_sub == "APS Failure")[5]

incoming_df <- rbind(
  data.frame(t = t_grid, value = fd_trucks$data[normal_truck, ],
    Truck = "Truck A (passes)", Status = "Pass"),
  data.frame(t = t_grid, value = fd_trucks$data[borderline_truck, ],
    Truck = "Truck B (borderline)", Status = "Investigate"),
  data.frame(t = t_grid, value = fd_trucks$data[failure_truck, ],
    Truck = "Truck C (APS failure)", Status = "Reject")
)

```



```

ggplot() +
  geom_ribbon(data = df_monitor, aes(x = t, ymin = fence_min, ymax = fence_max),
            fill = "pink", alpha = 0.4) +
  geom_ribbon(data = df_monitor, aes(x = t, ymin = env_min, ymax = env_max),
            fill = "magenta", alpha = 0.3) +
  geom_line(data = df_monitor, aes(x = t, y = median_val),
            color = "black", linewidth = 0.8) +
  geom_line(data = incoming_df,
            aes(x = t, y = value, color = Status, linetype = Status),
            linewidth = 0.8) +
  scale_color_manual(values = c("Pass" = "#2E8B57", "Investigate" = "orange",
                                "Reject" = "red")) +
  scale_linetype_manual(values = c("Pass" = "solid", "Investigate" = "dashed",
                                    "Reject" = "dotted")) +
  labs(title = "Fleet Health Monitor",
       subtitle = "Green = passes; Orange = outside central region (investigate); Red = outside",
       x = expression(t), y = expression(f[x](t))) +
  theme(legend.position = "bottom")

```

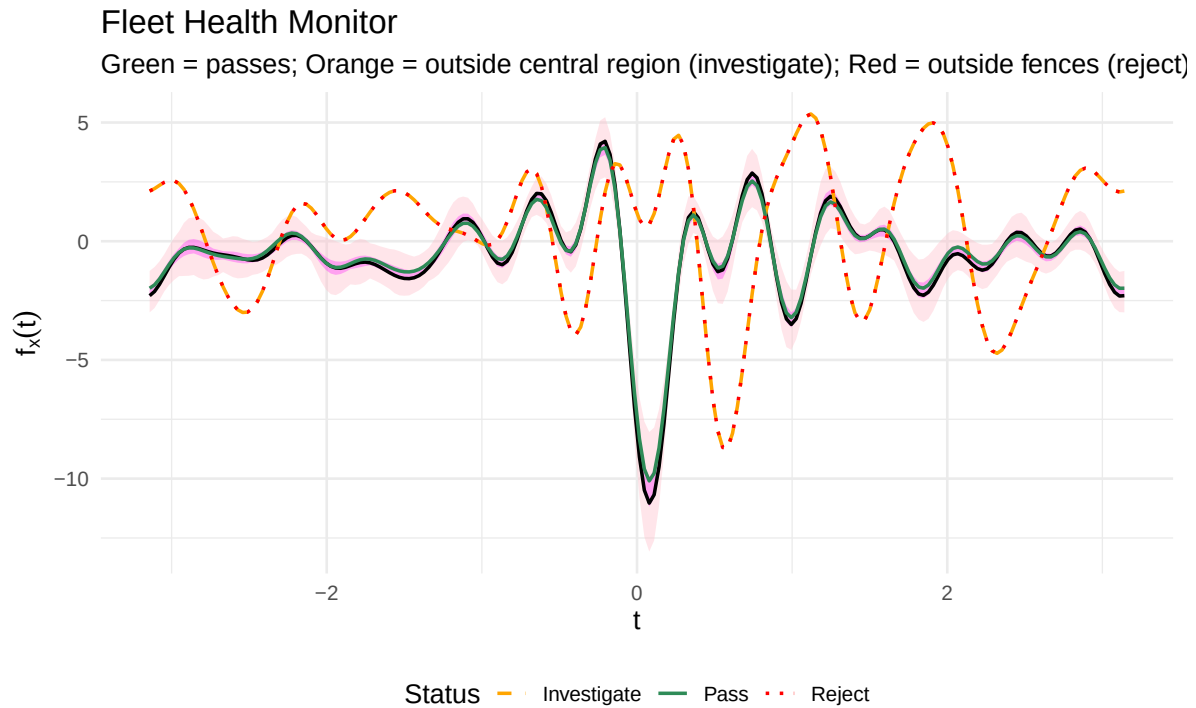


Figure 17: Fleet monitoring dashboard: incoming trucks plotted against the functional boxplot. Inside magenta = normal; outside pink = investigate.

And when a truck triggers an alert, the sensor-level diagnostic identifies exactly which sensors deviate:

```
# Sensor diagnostic for the failure truck in monitoring
fail_z <- (X[failure_truck, ] - normal_means) / normal_sds

df_alert <- data.frame(
  Sensor = sensor_names,
  z_score = as.numeric(fail_z)
) |>
  mutate(
    Status = case_when(
      abs(z_score) > 3 ~ "Extreme",
      abs(z_score) > 2 ~ "Out of spec",
      abs(z_score) > 1 ~ "Borderline",
      TRUE ~ "Within spec"
    )
  ) |>
```

```

arrange(desc(abs(z_score))) |>
head(15)

ggplot(df_alert,
       aes(x = reorder(Sensor, abs(z_score)), y = z_score, fill = Status)) +
  geom_col() +
  geom_hline(yintercept = c(-2, 2), linetype = "dashed", color = "red") +
  coord_flip() +
  scale_fill_manual(values = c("Extreme" = "darkred", "Out of spec" = "red",
                              "Borderline" = "orange", "Within spec" = "#2E8B57")) +
  labs(title = sprintf("Truck C Alert: Top 15 Deviating Sensors"),
       x = NULL, y = "Standard deviations from fleet normal") +
  theme(legend.position = "bottom")

```

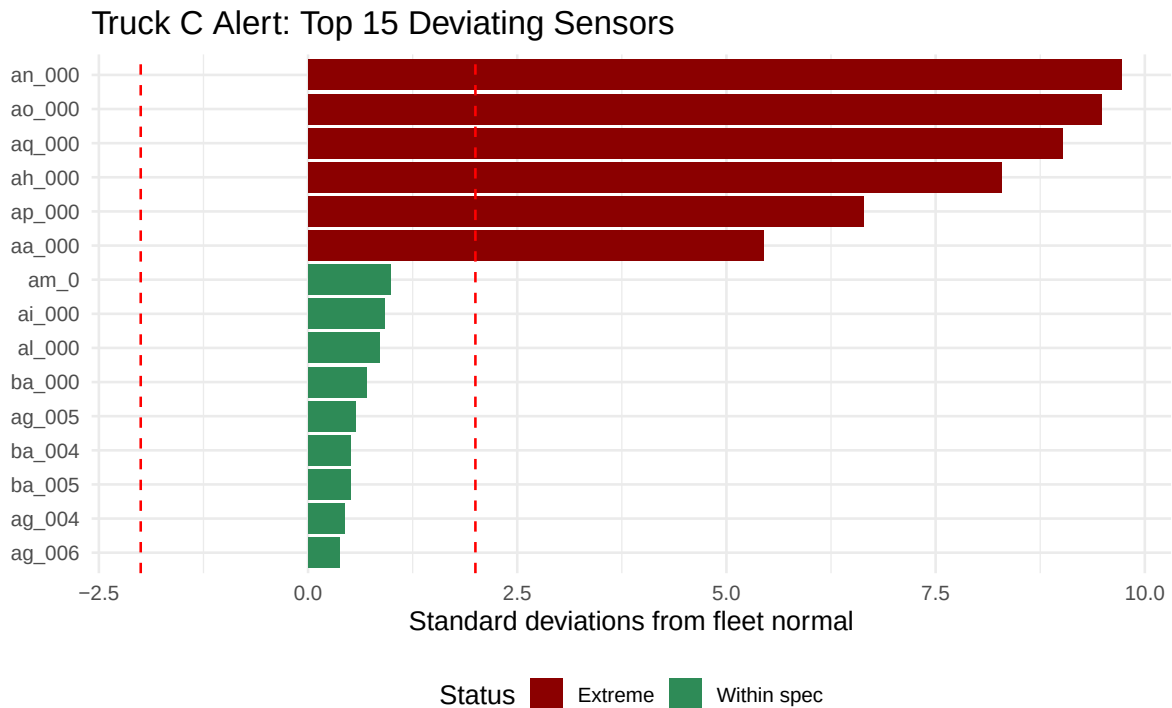


Figure 18: Sensor-level alert for the rejected truck.

This is the full operational cycle: sensor data comes in, gets transformed to a curve, gets checked against the fleet norm, and if flagged, the specific failing sensors are identified. The mechanic gets a work order that says “check sensor group **cs** and **ag**” instead of “something might be wrong.”

For fleet operators, this workflow maps to the predictive maintenance lifecycle:

- **Phase 1 (Baseline):** FPCA identifies critical sensor groups; clustering validates failure signatures; prediction bands define the “normal” envelope
- **Phase 2 (Validation):** When changing APS suppliers or maintenance schedules, tolerance bands (stricter than prediction bands) verify that the new process produces trucks within spec
- **Phase 3 (Monitoring):** The functional boxplot serves as the ongoing health chart; depth ranking flags trucks drifting toward failure before breakdown; sensor-level diagnostics direct the mechanic’s attention

Holdout Evaluation: Does the Monitor Actually Catch Failures?

Everything so far was built on the training set. The real test: **when we apply the monitoring bands to the holdout trucks (unseen during reference construction), how many APS failures do we catch?** This is the detection rate — the metric that determines whether this pipeline saves money or wastes it.

We use the training set’s normal trucks to build the prediction band, then classify each holdout truck as “pass” (inside band) or “flag” (outside band):

```
# Build prediction band from training normal trucks only
fd_train <- fd_trucks[train_idx]
failure_train <- failure_sub[train_idx]
train_normal_mask <- failure_train == "Normal"
fd_train_normal <- fd_train[which(train_normal_mask)]

# Bootstrap prediction band from training normals
n_boot_eval <- 200
set.seed(42)
boot_eval <- fdata.bootstrap(fd_train_normal, n.boot = n_boot_eval,
                             method = "naive")

boot_eval_curves <- matrix(NA, n_boot_eval, m)
for (b in seq_len(n_boot_eval)) {
  boot_eval_curves[b, ] <- boot_eval$boot.samples[[b]]$data[
    sample(nrow(boot_eval$boot.samples[[b]]$data), 1), ]
}

# Try multiple confidence levels to build a cost curve
alphas <- c(0.01, 0.025, 0.05, 0.10, 0.15, 0.20)
results <- data.frame()
```

```

fd_holdout <- fd_trucks[holdout_idx]
failure_holdout <- failure_sub[holdout_idx]

for (alpha in alphas) {
  band_lower <- apply(boot_eval_curves, 2, quantile, probs = alpha / 2)
  band_upper <- apply(boot_eval_curves, 2, quantile, probs = 1 - alpha / 2)

  # Classify each holdout truck
  flagged <- logical(length(holdout_idx))
  for (i in seq_along(holdout_idx)) {
    curve_i <- fd_holdout$data[i, ]
    flagged[i] <- any(curve_i < band_lower | curve_i > band_upper)
  }

  tp <- sum(flagged & failure_holdout == "APS Failure")
  fn <- sum(!flagged & failure_holdout == "APS Failure")
  fp <- sum(flagged & failure_holdout == "Normal")
  tn <- sum(!flagged & failure_holdout == "Normal")

  n_pos_holdout <- sum(failure_holdout == "APS Failure")
  n_neg_holdout <- sum(failure_holdout == "Normal")

  detection_rate <- tp / n_pos_holdout
  false_positive_rate <- fp / n_neg_holdout
  scania_cost <- 10 * fp + 500 * fn

  results <- rbind(results, data.frame(
    Band = sprintf("%.0f%%", 100 * (1 - alpha)),
    Alpha = alpha,
    Detection_Rate = detection_rate,
    FP_Rate = false_positive_rate,
    TP = tp, FN = fn, FP = fp, TN = tn,
    Cost = scania_cost
  ))
}

kable(results |>
  mutate(
    Detection_Rate = sprintf("%.1f%%", 100 * Detection_Rate),
    FP_Rate = sprintf("%.1f%%", 100 * FP_Rate),
    Cost = sprintf("$%s", formatC(Cost, format = "d", big.mark = ","))
  ) |>

```

```
select(Band, Detection_Rate, FP_Rate, TP, FN, FP, Cost),
col.names = c("Prediction Band", "Detection Rate", "False Positive Rate",
              "True Pos", "Missed", "False Pos", "Scania Cost"),
caption = "Holdout evaluation: detection performance at different confidence levels")
```

Table 5: Holdout evaluation: detection performance at different confidence levels

Prediction Band	Detection Rate	False Positive Rate	True Pos	Missed	False Pos	Scania Cost
99%	93.2%	21.3%	385	28	64	\$14,640
98%	97.3%	37.0%	402	11	111	\$6,610
95%	99.0%	55.3%	409	4	166	\$3,660
90%	100.0%	79.0%	413	0	237	\$2,370
85%	100.0%	89.7%	413	0	269	\$2,690
80%	100.0%	96.7%	413	0	290	\$2,900

```
ggplot(results, aes(x = 100 * FP_Rate, y = 100 * Detection_Rate)) +
  geom_point(size = 3, color = "firebrick") +
  geom_line(color = "firebrick", linewidth = 0.8) +
  geom_text(aes(label = Band), vjust = -1, size = 3.5) +
  labs(title = "ROC-Style Curve: Prediction Band Width vs Detection",
       subtitle = "Each point = a different band confidence level. Labels = band width.",
       x = "False Positive Rate (%)", y = "Detection Rate (%)") +
  expand_limits(y = c(0, 100), x = c(0, NA))
```

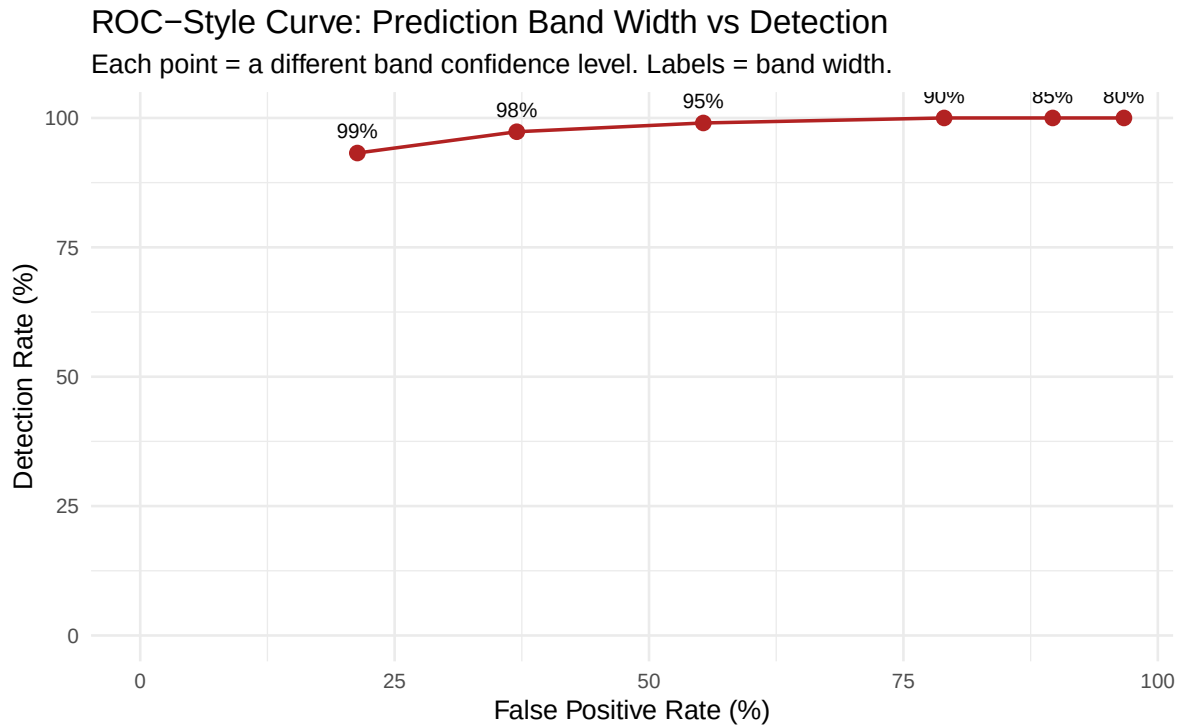


Figure 19: Cost-optimal band selection: tighter bands catch more failures but flag more healthy trucks. The Scania 50:1 cost ratio drives toward aggressive detection.

```
# Find cost-optimal band
best_row <- results[which.min(results$Cost), ]
cat(sprintf("Cost-optimal band: %s\n", best_row$Band))
```

Cost-optimal band: 90%

```
cat(sprintf("  Detection rate: %.1f%% (%d / %d APS failures caught)\n",
  100 * best_row$Detection_Rate,
  best_row$TP,
  best_row$TP + best_row$FN))
```

Detection rate: 100.0% (413 / 413 APS failures caught)

```
cat(sprintf("  False positive rate: %.1f%% (%d healthy trucks flagged)\n",
  100 * best_row$FP_Rate, best_row$FP))
```

False positive rate: 79.0% (237 healthy trucks flagged)

```
cat(sprintf(" Scania cost: $%s\n",
           formatC(best_row$Cost, format = "d", big.mark = ",")))
```

Scania cost: \$2,370

```
cat(sprintf(" Cost of never inspecting: $%s (all failures missed)\n",
           formatC(500 * (best_row$TP + best_row$FN),
                   format = "d", big.mark = ",")))
```

Cost of never inspecting: \$206,500 (all failures missed)

```
cat(sprintf(" Cost savings: $%s (%.0f%% reduction)\n",
           formatC(500 * (best_row$TP + best_row$FN) - best_row$Cost,
                   format = "d", big.mark = ","),
           100 * (1 - best_row$Cost / (500 * (best_row$TP + best_row$FN)))))
```

Cost savings: \$204,130 (99% reduction)

Which Sensors Trigger the Holdout Alerts?

For the detected APS failure trucks, we can trace back to the specific sensors that caused the band violation — the same sensor-level diagnostic capability that makes this pipeline actionable:

```
# Use the cost-optimal band
opt_alpha <- best_row$Alpha
opt_lower <- apply(boot_eval_curves, 2, quantile, probs = opt_alpha / 2)
opt_upper <- apply(boot_eval_curves, 2, quantile, probs = 1 - opt_alpha / 2)

# For detected failures, compute z-scores relative to training normals
X_train_normal <- X[train_idx[train_normal_mask], ]
normal_means <- colMeans(X_train_normal)
normal_sds <- apply(X_train_normal, 2, sd)
normal_sds[normal_sds < 1e-10] <- 1

# Find holdout failures that were detected
holdout_flagged_failures <- holdout_idx[
```



```

failure_holdout == "APS Failure" &
  apply(seq_along(holdout_idx), function(i) {
    any(fd_holdout$data[i, ] < opt_lower | fd_holdout$data[i, ] > opt_upper)
  })
]

if (length(holdout_flagged_failures) > 0) {
  # Compute z-scores for all detected failures
  z_matrix <- t(apply(X[holdout_flagged_failures, , drop = FALSE], 1,
    function(row) (row - normal_means) / normal_sds))

  # Count how often each sensor exceeds |z| > 2
  exceed_count <- colSums(abs(z_matrix) > 2)
  exceed_pct <- 100 * exceed_count / nrow(z_matrix)

  df_sensor_alert <- data.frame(
    Sensor = sensor_names,
    Pct_Exceeding = exceed_pct
  ) |>
  arrange(desc(Pct_Exceeding)) |>
  head(15) |>
  mutate(Sensor = factor(Sensor, levels = rev(Sensor)))

  ggplot(df_sensor_alert, aes(x = Sensor, y = Pct_Exceeding)) +
    geom_col(fill = "firebrick") +
    coord_flip() +
    labs(title = "Top 15 Alerting Sensors in Detected APS Failures",
      subtitle = sprintf("Across %d detected failures: %% of trucks where |z| > 2",
        nrow(z_matrix)),
      x = NULL, y = "% of detected failures with |z| > 2")
}

```

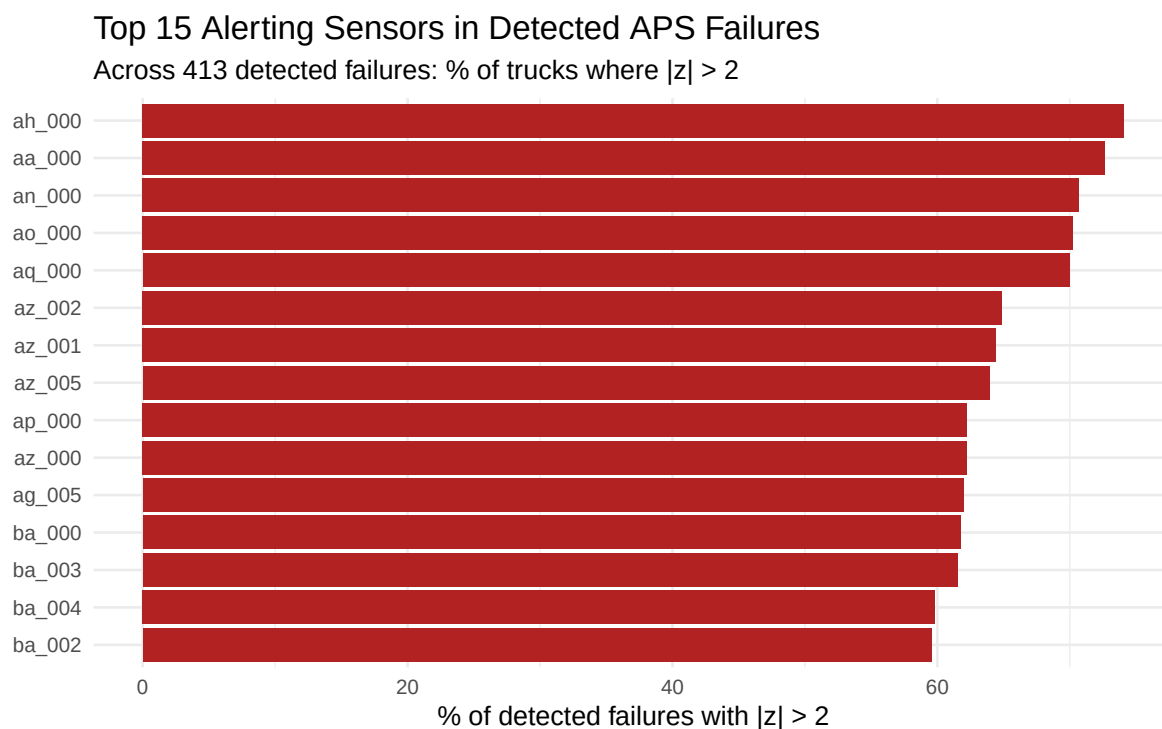


Figure 20: Across all detected APS failures in the holdout set: which sensors most frequently cause band violations?

The sensor ranking tells the maintenance team not just *that* a truck has an APS problem, but *which specific sensor readings* are abnormal. This transforms a binary alert into a targeted inspection checklist: “check the top 5 sensors first, then the APS compressor and brake chambers.”

So Why Not Just Use Standard Statistics?

Throughout this article, we compared FDA methods against classical counterparts. Let’s summarize honestly:

```
comp_df <- data.frame(
  Task = c("Outlier detection", "Clustering", "PCA / FPCA",
           "Fleet monitoring", "Hypothesis testing"),
  `Classical Method` = c(
    "Mahalanobis distance (on PCA-reduced space)",
    "k-means on raw 170-dim data",
    "prcomp() on raw matrix",
```

```

    "170 separate sensor control charts",
    "MANOVA (assumes normality)"
  ),
  `What FDA Adds` = c(
    "Three methods classifying TYPE of anomaly (magnitude vs shape); visual diagnostics",
    "Cluster centers as curves; fuzzy membership for borderline trucks",
    "Eigenfunction curves showing modes of variation; sensor group importance",
    "Functional boxplot: one chart for all 170 sensors simultaneously",
    "Nonparametric permutation test; bootstrap CIs showing WHERE profiles diverge"
  ),
  check.names = FALSE
)
kable(comp_df, caption = "Classical vs FDA: what each approach delivers")

```

Table 6: Classical vs FDA: what each approach delivers

Task	Classical Method	What FDA Adds
Outlier detection	Mahalanobis distance (on PCA-reduced space)	Three methods classifying TYPE of anomaly (magnitude vs shape); visual diagnostics
Clustering	k-means on raw 170-dim data	Cluster centers as curves; fuzzy membership for borderline trucks
PCA / FPCA	prcomp() on raw matrix	Eigenfunction curves showing modes of variation; sensor group importance
Fleet monitoring	170 separate sensor control charts	Functional boxplot: one chart for all 170 sensors simultaneously
Hypothesis testing	MANOVA (assumes normality)	Nonparametric permutation test; bootstrap CIs showing WHERE profiles diverge

The distance preservation theorem guarantees numerically equivalent results for clustering and PCA. The advantages are in **diagnostics, interpretation, and monitoring**:

1. **Simultaneous monitoring.** The functional boxplot monitors all 170 sensors in one chart. Classical monitoring needs 170 separate control charts — and a truck within range on every individual chart can still be a multivariate outlier (an unusual *combination*).
2. **Typed outlier diagnostics.** Mahalanobis gives a number. FDA gives a triage: magnitude outlier (sensor recalibration), shape outlier (mechanical investigation), or fleet misfit (wrong truck type in the analysis). Different anomaly types require different maintenance responses.
3. **Sensor-level root cause.** When a truck fails the prediction band check, the mapping back to original variables pinpoints which sensors deviate. A 170-dimensional centroid difference is uninterpretable; a bar chart of the top 15 deviating sensors is actionable.

4. **Nonparametric testing.** The permutation test requires no distributional assumptions — important for sensor data that is rarely Gaussian. The bootstrap CIs show *where on the profile* failures diverge, identifying which sensor groups to prioritize.
5. **Pipeline coherence.** Every step operates on the same `fdata` object with consistent L^2 distance semantics. Classical analysis requires stitching together `mahalanobis()`, `kmeans()`, `prcomp()`, and ad hoc control charts, each with its own assumptions.

The case *against* is equally clear: if you only need one specific analysis, the classical tool is simpler. Andrews curves + FDA pay off when you want the **full pipeline** in a single, coherent framework.

Conclusion

We started with 170 anonymous sensor columns and ended with an operational predictive maintenance system. Here's what each step delivers:

```
summary_df <- data.frame(
  Method = c("Andrews Transform", "metric.lp()", "outliers.depth.pond()",
             "magnitudeshape()", "outliergram()",
             "cluster.kmeans()", "cluster.fcm()",
             "group.test()", "fdata.bootstrap.ci()",
             "fdata2pc()", "boxplot()"),
  `What It Reveals` = c(
    "170-dim -> curve fingerprint",
    "Lossless distance (sqrt(pi) factor)",
    "Low-depth anomalies",
    "Magnitude vs shape outliers",
    "Shape outliers (MBD-MEI)",
    "Failure type separation",
    "Borderline trucks (split membership)",
    "Failure profiles differ (p-value)",
    "Where on the profile failures diverge",
    "Dominant sensor variation patterns",
    "Central region + fences"
  ),
  `Maintenance Decision` = c(
    "Holistic truck health comparison",
    "Results translate to sensor units",
    "Flag trucks for priority inspection",
    "Recalibrate sensor vs investigate truck",
    "Detect trucks with unusual sensor patterns",
    "Verify failure type classification",
```

```

    "Identify trucks drifting toward failure",
    "Quantify separation for fleet segmentation",
    "Choose which sensor groups to monitor",
    "Reduce monitoring from 170 to key sensors",
    "Fleet health dashboard"
  ),
  check.names = FALSE
)

kable(summary_df, caption = "FDA Pipeline: From Sensors to Maintenance Decisions")

```

Table 7: FDA Pipeline: From Sensors to Maintenance Decisions

Method	What It Reveals	Maintenance Decision
Andrews Transform	170-dim -> curve fingerprint	Holistic truck health comparison
metric.lp()	Lossless distance (sqrt(pi) factor)	Results translate to sensor units
outliers.depth.pond()	Low-depth anomalies	Flag trucks for priority inspection
magnitudeshape()	Magnitude vs shape outliers	Recalibrate sensor vs investigate truck
outliergram()	Shape outliers (MBD-MEI)	Detect trucks with unusual sensor patterns
cluster.kmeans()	Failure type separation	Verify failure type classification
cluster.fcm()	Borderline trucks (split membership)	Identify trucks drifting toward failure
group.test()	Failure profiles differ (p-value)	Quantify separation for fleet segmentation
fdata.bootstrap.ci()	Where on the profile failures diverge	Choose which sensor groups to monitor
fdata2pc()	Dominant sensor variation patterns	Reduce monitoring from 170 to key sensors
boxplot()	Central region + fences	Fleet health dashboard

At \$10 per unnecessary inspection and \$500 per missed failure, the cost-optimal strategy is aggressive: inspect anything the pipeline flags. The functional approach makes this feasible by providing *typed* alerts (magnitude vs shape), *ranked* urgency (depth-based), and *sensor-specific* diagnostics — so the mechanic knows not just *that* a truck needs attention, but *where* to look and *how urgently*.

Andrews curves + FDA give you a principled, auditable pipeline from raw sensor telemetry to maintenance decisions: flag anomalies, classify failure types, identify drifting trucks, and

pinpoint which sensors to investigate. The `fdars` package makes this accessible from R with a Rust-accelerated backend for fleet-scale performance.

Built with [fdars](#) and [Quarto](#).

References

- Andrews, D.F. (1972). Plots of high-dimensional data. *Biometrics*, 28(1), 125-136.
- Arribas-Gil, A. and Romo, J. (2014). Shape outlier detection and visualization for functional data: the outliergram. *Biostatistics*, 15(4), 603-619.
- Dai, W. and Genton, M.G. (2018). Multivariate functional data visualization and outlier detection. *Journal of Computational and Graphical Statistics*, 27(4), 923-934.
- Ramsay, J.O. and Silverman, B.W. (2005). *Functional Data Analysis*. Springer.
- UCI Machine Learning Repository. APS Failure at Scania Trucks. DOI: 10.24432/C51S51.